



TM251

Memory organization and IEC programming

Requirements

Training modules	TM210 - Automation Studio TM246 - Structured Text (ST)
Software	Automation Runtime 6.0 Automation Studio 6.1
Hardware	ArSim / X20CP1686x

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	4
1.2 Symbols and safety notices.....	4
2 Memory.....	5
2.1 Number systems.....	5
2.2 Memory organization.....	8
2.3 Data formats.....	9
3 Variables, constants and data types.....	11
3.1 Basic information.....	11
3.2 Derived data types.....	16
3.3 Memory requirements of variables.....	21
3.4 Dynamic memory access (pointers).....	24
4 Arrays and strings.....	29
4.1 Arrays.....	29
4.2 Strings.....	34
4.3 Operations on memory areas.....	39
4.4 Error handling with IEC Check.....	41
5 Advanced memory management.....	48
5.1 Dynamic memory.....	48
6 Summary.....	51

1 Introduction

For correct use of variables and data types, it is important to have a basic understanding of how the memory on a controller is managed. Knowledge of how the memory is organized and the concept of addressing memory cells, blocks or whole areas is indispensable to be able to create an error-free application.

The use of user data types and constants improves the flexibility and consistency of the application. The use of dynamic memory¹ ensures optimum use of resources and high-performance.

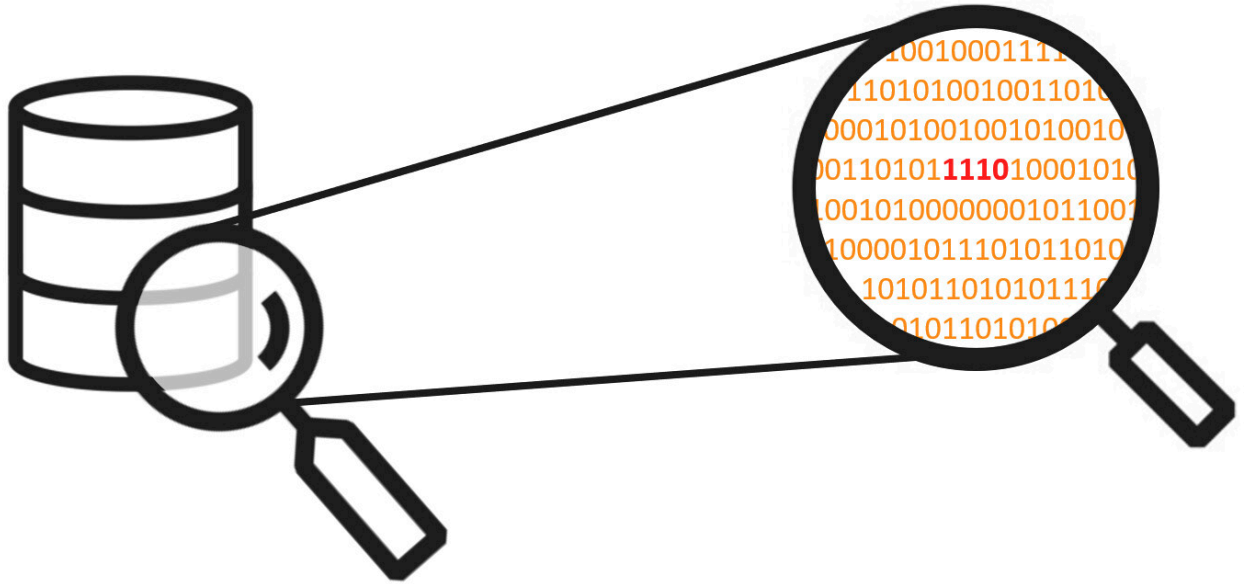


Figure 1: Correct memory organization is the basis of every executable application

This training module covers IEC data types, applying them correctly and getting an understanding of how memory can be used in and by an application. Topics range from the representation of binary information to data types and the use of dynamic memory.

The focus is on avoiding errors and being able to analyze and eliminate any software errors that may occur.

1.1 Learning objectives

This training module uses selected application examples and exercises to help participants learn the basics of memory organization and management on the controller.

Detailed learning objectives:

- Learning about the memory organization of B&R controllers
- Learning to use both simple and complex data types correctly and confidently.
- Explaining the dangers that can occur when using arrays, strings and pointers.
- Deliberately creating error situations through the improper use of arrays, strings and pointers.
- Using different tools to find the causes of memory access violations.

1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 - Working with Automation Studio" apply.

¹ The term "dynamic memory" should be qualified in this context because on B&R control systems only the use of the memory is dynamic and not its allocation.

2 Memory

Like most computer systems,² a control system also has different types of memory for storing data. A distinction is made between **ROM**³ (nonvolatile) and **RAM** (volatile).

Data can be managed on the controller in many different ways. It can be stored as process variables in RAM, on a USB flash drive, as a file in a file system or as a network file.

During startup of the controller, Automation Runtime copies all system and user tasks, as well as configured components, to DRAM (a sub-area of RAM) and executes them there. Accessing DRAM is faster than accessing flash memory (ROM).



[Automation Runtime \ Method of operation \ Memory](#)

The examples and exercises shown in this document refer to information that is in DRAM at runtime (Automation Runtime). The memory addresses mentioned as well as their values also originate from the DRAM.

2.1 Number systems

A computer system can only store and process data (files, programs, variables, recipes, measurement data) in binary form, i.e. as a sequence of zeros and ones in memory. The zeros and ones can be represented in hardware either by two distinguishable magnetic states (hard disks) or two electrical states (flash memory, RAM).

2.1.1 Binary system

In a binary computer system, information is represented as ones and zeros. Understanding the basics of binary logic makes programming and debugging a system easier.

The decimal system (base-10 system) used by people in everyday life has a digit set of ten digits (0 to 9). In contrast, the binary system (base-2 system) only has two digits (0 and 1). As with the decimal system, each position (place) in a binary number has a value⁴.

Representation of a binary number

A binary number has a certain number of positions. The example shows a number with eight positions. The positions are always counted from the right starting with index 0.

Each position has a certain value. The value of a position is calculated from 2^{index} .

Index 0 (2^0) thus has the value 1. And index 4 has the value $2^4 = 16$.

128	64	32	16	8	4	2	1	value (2^{index})
7	6	5	4	3	2	1	0	index of binary number

Figure 2: Correlation between the position (place) and value of a binary number

Each position of a binary number can contain either a 0 or a 1. If there is a 1, the value of the position is included. If there is a 0, the value of the position is not added to the result.

² A digital system that contains hardware and software and can therefore perform complex tasks.

³ Although today, with the use of flash memory, it is no longer strictly speaking ROM (read-only memory).

⁴ Known from the decimal system by the terms "ones place", "tens place", "hundreds place" and so on.

Example of a binary number

To obtain the decimal value of a binary number, all values of positions containing a 1 are added up.

In the example, this means $128 + 16 + 8 + 4 + 1 = 157$

128	64	32	16	8	4	2	1	value (2^{index})
1	0	0	1	1	1	0	1	
7	6	5	4	3	2	1	0	index

Figure 3: Binary representation of the decimal number 157

Length of a binary number

The maximum value of an eight-digit binary number is 255. A total of 256 different numbers can be represented (0-255).

A binary digit corresponds to the smallest possible unit of information on a computer system. This smallest unit of information is called a bit and can only contain the values 0 and 1.

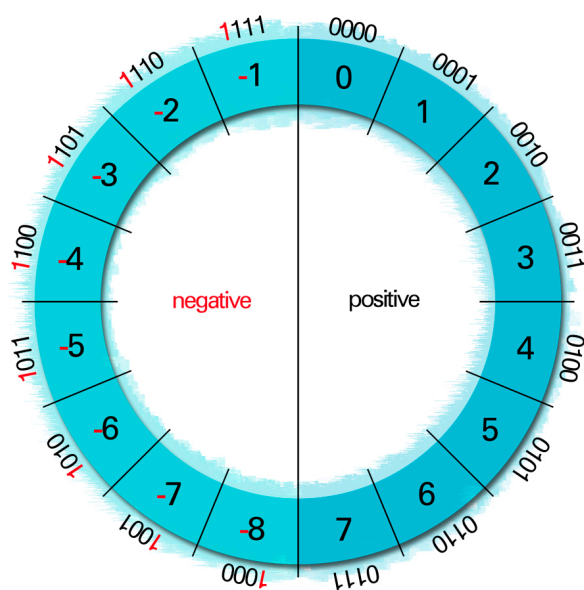
In computer systems, information is always stored in multiples of 8 bits (i.e. 1 byte), which is described in [2.2 "Memory organization" on page 8](#) in more detail.

Therefore, it may happen that more digits (bits) are used to represent the binary number than would be necessary. These digits are filled with leading zeros.

128	64	32	16	8	4	2	1	value (2^{index})
0	0	0	0	1	1	0	1	
7	6	5	4	3	2	1	0	index

Figure 4: Binary representation of the decimal number 13

Sign of a binary number



In binary, the bit with the highest value (i.e. the bit on the far left) is used as the sign bit. As a result, 7 bits are left to represent the value. (In the figure on the left, a binary number is represented with only 4 bits for reasons of clarity, so 3 bits are left to represent the value.)

Negative numbers are formed using two's complement.

- 1) The bit pattern is created from the positive decimal number.
- 2) This bit pattern is then inverted.
- 3) 1 is added.⁵

The result is then the bit pattern for the negative number.

Figure 5: Representation of negative numbers in the binary system (4-bit)



Binary representation of -3 (8-bit)

1.	0000 0011	Binary representation of +3
2.	1111 1100	All bits are inverted
3.	1111 1101	Binary addition of 0000 0001
	1111 1101	Binary representation of -3

⁵ The number zero is the only number that has no sign. For this reason, 1 must be added after inverting.



Using a sign bit shifts the value range of a number.
For the example of an 8-bit number, this means:

Value range without sign	Value range with sign
0 to 255	-128 to +127

The total quantity of different numbers that can be represented is always 256 in both cases.

This is especially important when signed and unsigned data types are mixed in programs: [2.3 "Data formats" on page 9](#)

2.1.2 Hexadecimal system

Binary data or bit patterns become difficult to read especially with longer sequences of numbers. The hexadecimal system is used instead because it provides a compact and clear representation of binary information.

Unlike the binary system, the hexadecimal system has a set of 16 digits: From 0 to 9 and A to F.

Hexadecimal digits	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal equivalent	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Representation of a hexadecimal number

In the hexadecimal system, a value is assigned for each position (as with the binary system). The value is calculated from 16^{index} .

4096	256	16	1	value (16^{index})
3	2	1	0	index

Example of a hexadecimal number

The total value is calculated by multiplying the value of the position by the digit there (for all positions).

In the example this means

$$(9 * 16) + (D=13 * 1) = 157$$

4096	256	16	1	value (16^{index})
0	0	9	D	
3	2	1	0	index

The common representation of hexadecimal numbers for computer systems and most programming languages is a leading "0x". For the example above, the representation is 0x9D. This prefix tells the compiler: "Here comes a hex number"⁶.

Correlation between binary and hexadecimal numbers

A hexadecimal position can be one of 16 digits (0-F). For a binary number to represent 16 different values, it needs four positions. This means that 4 binary positions can be combined into one hexadecimal position.

Binary	0101	1100	0011	1111
Hexadecimal	5	C	3	F

⁶ In the ST programming language, a "16#" is required instead of the preceding "0x"

Conversion table for 4-bit

Binary	Hexadecimal	Decimal		Binary	Hexadecimal	Decimal
0000	0	0		1000	8	8
0001	1	1		1001	9	9
0010	2	2		1010	A	10
0011	3	3		1011	B	11
0100	4	4		1100	C	12
0101	5	5		1101	D	13
0110	6	6		1110	E	14
0111	7	7		1111	F	15

A byte consists of 8 bits, i.e. 8 binary positions, and a byte can also be represented by 2 hexadecimal digits.



Hexadecimal representation of numbers is primarily used when Logger entries or addresses are displayed.

The Watch window offers the option of displaying variable values in binary, octal, decimal or hexadecimal.

IEC 61131-3 specifies the prefixes 2# (e.g. 2#0000_1001) for binary and 16# (e.g. 16#09) for hexadecimal.



[Programming \ Standards \ Literals in IEC languages](#)

2.2 Memory organization

address

0x0471DC7E

memory cell

10001011

The memory of a system is divided into memory cells, each containing 1 byte (i.e. 8 bits). The system can address each memory cell by its unique address.

Figure 6: Memory cell with address

Correlation between memory cell, address, variable name and value

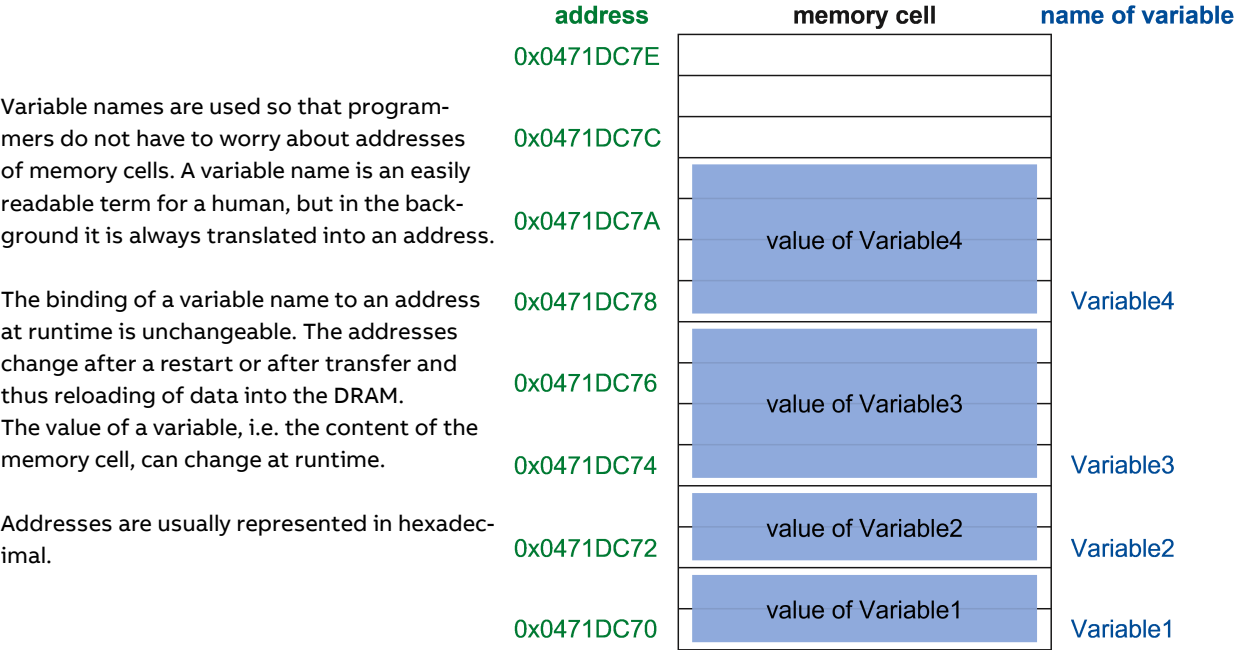


Figure 7: Correlation between memory cell, variable name, address and value



On a 32-bit system, addresses are 32 bits long. The maximum amount of memory that can be managed by the system automatically results from the length of the addresses, since each memory cell must have a unique address. This limits the maximum possible number of memory cells.

If an address must be stored, this requires a variable with a data type that can hold 32 bits, i.e. 4 bytes. According to IEC 61131-3,⁷ this applies to UDINT.

2.3 Data formats

The binary data in memory alone is not enough to generate correct information from it. In addition, knowledge of how to read or interpret the binary data is required. This is called data format.

In the simplest case, this means knowing the data type.

Interpretation as an unsigned variable (USINT ⁸)										Interpretation as a signed variable (SINT ⁹)									
address					memory cell					address					memory cell				
0x0471DC7E					10001011					0x0471DC7E					10001011				
Returns the value 139										Returns the value -117									

Table 1: Different interpretation of the same memory content

7 IEC 61131-3 is abbreviated to IEC in the following sections.

8 USINT stands for Unsigned Short INT and is a 1-byte IEC data type

9 SINT stands for Short INT and is also a 1-byte IEC data type

Interpretation as an unsigned variable (USINT ⁸)	Interpretation as a signed variable (SINT ⁹)
The result is obtained by adding all the values (128+8+2+1).	The bit on the far left is the sign bit. The (negative) decimal value can be derived by forming the two's complement.

Table 1: Different interpretation of the same memory content

Even more information is needed on a higher level (e.g. files) to interpret the stored (binary) data correctly. Without knowing the appropriate data format, the data is just raw binary data that cannot be processed.

	Data format	Binary data (hexadecimal notation)
Binary	2B 34 33 37 37 34 38 36 35 38 36	2B 34 33 37 37 34 38 36 35 38 36
Data type REAL	12 : 34	41 43 D7 0A
ASCII	'Hello World!'	48 65 6C 6C 6F 20 57 6F 72 6C 64 21 00
XML	<?xml version="1.0"> <ComboBox> <Item ID="off"/> </ComboBox>	3C 3F 78 6D 6C 20 76 65 72 73 69 6F 6E 3D 22 31 2E 30 22 3E 0D 0A 3C 43 6F 6D 62 6F 42 6F 78 3E 0D 0A 3C 49 74 65 6D 20 49 44 3D 22 6F 66 66 22 2F 3E 0D 0A 3C 2F 43 6F 6D 62 6F 42 6F 78 3E 0D 0A

Table 2: Comparison of data format and binary representation



As can be seen in the table, all data is always stored in binary. It can only be represented differently if the data format is known. A file extension tells nothing about the contents of a file; it simply provides a clue about how the data should be interpreted.

8 USINT stands for Unsigned Short INT and is a 1-byte IEC data type

9 SINT stands for Short INT and is also a 1-byte IEC data type

3 Variables, constants and data types

Programming is not carried out by accessing fixed memory addresses, but via symbolic elements that have names. These elements are called process variables.

3.1 Basic information

If data is managed in the form of variables on a system, both the identifier (variable name) and the data type must be made known to the compiler. Only this allows correct transformation of the source code and the corresponding memory organization on the target system.

3.1.1 Overview of IEC 61131-3 base data types

A data type must be specified at the declaration for each variable and constant. Basic data types are also called base data types. These data types form the basis for all other derived data types.

The following list represents all available base data types according to IEC 61131-3 as well as their possible uses. The defined memory space according to the IEC standard is listed for each data type.

Binary / Bit string		Signed integer		Unsigned integers		Floating-point number		Character string	Time, date
BOOL	1-bit ¹⁰	SINT	8-bit	USINT	8-bit	REAL	32-bit	STRING ¹¹	TIME ¹²
BYTE	8-bit	INT	16-bit	UINT	16-bit	LREAL	64-bit	WSTRING	DATE_AND_TIME (DT)
WORD	16-bit	DINT	32-bit	UDINT	32-bit				DATE
DWORD	32-bit								TIME_OF_DAY (TOD)

Table 3: Overview of IEC data types

A complete list of base data types, their areas of use and range of values can be found in Automation Help.



The data types BYTE, WORD and DWORD are pure bit strings. Arithmetic operations are therefore not allowed by the compiler.



For floating-point data types (REAL, LREAL), an exact comparison of two values is not possible because for these data types only approximate values and never exact values can be stored.



[Programming \ Variables and data types \ Data types](#)

3.1.2 Declaring variables and constants

Declaration

The variable declaration tells the compiler the name of a variable and its data type.

Variables and constants are generally only created in files with the extension .var. When adding a new program, a file for variable declaration is automatically created as well.

¹⁰ Since the smallest possible memory unit is 8 bits, 8 bits are also used by a BOOL in memory.

¹¹ Each single character of a STRING uses 8 bits of memory, one character of a WSTRING uses 16 bits, see chapter 4.2.1 "Declaring strings" on page 34.

¹² All time and date data types are managed in the background as UDINT and thus using 32 bits.

In addition, the Toolbox offers the possibility to insert further .var files at the desired position in the project structure.

Initialization

Initializing is the process of assigning an initial value to a variable.

Initializing variables can take place either directly in the declaration editors or in the initialization subroutine of the program. Caution is advised when using remanent data. The initialization of a remanent variable in the initialization subroutine means that the remanent data is also reinitialized during a warm restart. For this reason, remanent variables should only be assigned a start value in the declaration editor.

Constants can only be "initialized" in the variable declaration file (.var file).



[Programming \ Editors \ Table editors \ Declaration editors \ Table editor for variable declaration](#)
[Programming \ Editors \ General operation \ Smart Edit](#)

Scope

Variables (and self-created data types) are only known within their scope. The scope initially depends on the position of the declaration file in the Logical View.

1) Global

Variables from "Global.var" are known in all programs and packages.

2) Package-global

The validity of "PackageVariables.var" initially refers only to all programs from "Package" as well as all underlying packages.

By right-clicking on the corresponding declaration file, the scope of "Package-global" can also be extended to "Global" under "Properties". This is of great advantage when structuring projects.

3) Local

A local variable from "Program" cannot be used outside the local task. The same also applies to "Program1" and "Program2".

If a variable is declared locally with a name that already exists globally, the program only has access to the locally declared variable. However, this should be avoided for reasons of traceability.

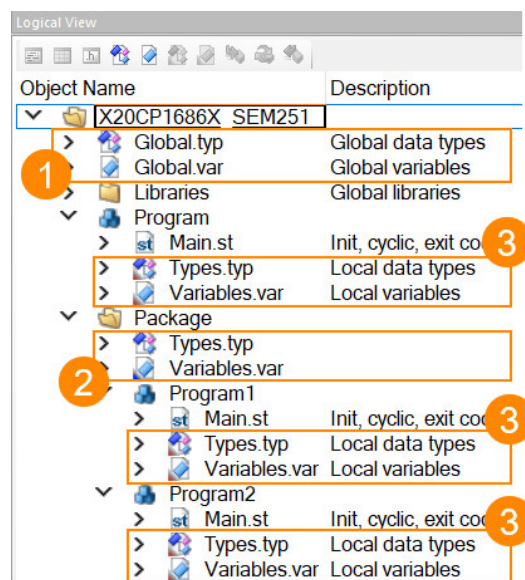


Figure 8: Declaration files with different scopes



[Programming \ Variables and data types \ Variables \ Scope of variables](#)

Declaring and initializing in Automation Studio

Variable declarations are displayed in this training module using the IEC text format.

Automation Studio users can choose between opening declarations in the table editor or text editor. When double-clicking on a declaration file (.var), the table editor is opened by default.

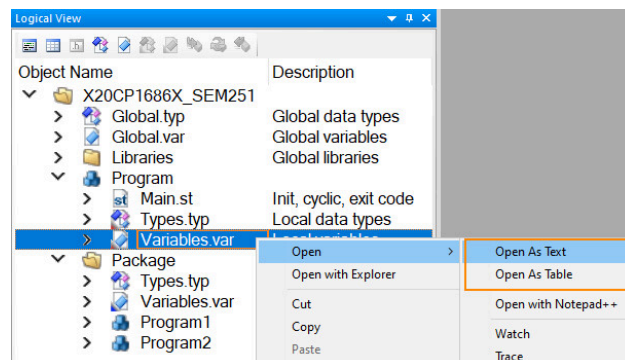


Figure 9: Open declaration as text

Depending on the selected method of representation, the declaration is displayed as a table (left) or as text (right).

 The screenshot shows a table titled 'Program::Variables.var [Variable Declaration]'. The table has five columns: 'Name', 'Type', 'Co...' (likely 'Constant'), 'Value', and 'Retain'. There are three rows of data:

Name	Type	Co...	Value	Retain
VariableUSINT	USINT	☐		☐
VariableSINT	SINT	☐	13	☐
MAX_INDEX	USINT	☒	0	☐

Figure 10: Variable declaration in table format

 The screenshot shows the same variable declaration in text format. The code is as follows:


```

1  VAR
2      VariableUSINT : USINT;
3      VariableSINT : SINT := 13;
4  END_VAR
5  VAR CONSTANT
6      MAX_INDEX : USINT := 0;
7  END_VAR
  
```

Figure 11: Variable declaration in text format

Constants

Like variables, constants have a name and a data type and are also stored in memory like variables.

Variables can get new values at runtime. Some examples of variables include digital and analog inputs/outputs as well as process data.

In contrast, the value of a constant cannot be modified by its name at runtime. Constants must already be initialized during the declaration. They are used in the program code as limit values, for example.

Using constants makes the source code easier to read and maintain.

Exercise: Declaring and using variables and constants

1) Declaring variables and a constant

Declare the variables `VariableUSINT` and `VariableSINT` with the appropriate data types. Initialize `VariableUSINT` with the value 137. Also declare the constant `MAX_INDEX` of type `USINT` and assign the value 10.

2) Representing values in binary and hexadecimal

Assign the value -119 to `VariableSINT` in the INIT part of the program or in the Watch window. Now view the two variables in the Watch window in binary and hexadecimal view.

3) Assigning a value to the constant

Assign the new value 42 to the constant in the program code.

4) Evaluating the output of the compiler

Compile the program and analyze the output from the compiler.



Assigning values to constants is not possible in the program code. The compiler will report the following error in the message window:

All ✖ 1 Error ⚠ 4 Warnings ! 25 Messages				
#	Category	Date/Time	Description	Error
10	✖ Error	20.05.2025 10:06:31,7853	No write access to variable 'MAX_INDEX' allowed.	1138

Constants can subsequently be used for initializing variables or array limits. In order to do this, however, the project setting "Allow extension of IEC standards" needs to be enabled.

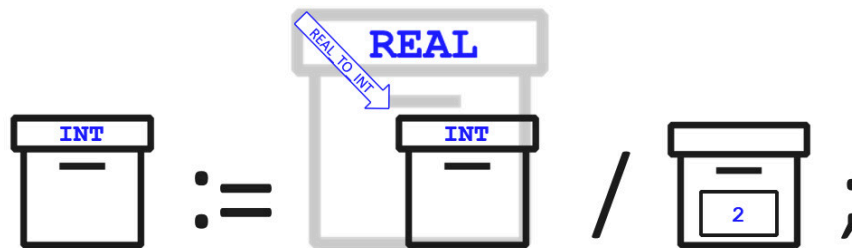


[Programming \ Variables and data types \ Variables \ Constants](#)

[Project management \ Workspace \ General project settings \ Settings for IEC compliance](#)

3.1.3 Data type conversion

During programming, it may become necessary to convert one data type to another. In general, this is always the case when two variables or values of different data types occur in the same expression. Caution is also advised if the value range of a data type would be exceeded by an addition.



```
VariableINT      := REAL_TO_INT(VariableREAL) / 2;
```

Figure 12: Different data types occurring in one expression

A distinction is made between implicit and explicit data type conversion.

Implicit data type conversion

Implicit data type conversion is a type of conversion that is performed automatically by the compiler. If the compiler finds data types with different accuracy or different range of values in a statement, it performs a conversion to the data type with the larger range of values.

If the data type of the result variable is too small to accommodate the result of the expression, implicit conversion is not possible and a compilation error is output.



Assigning a data type with a smaller range of values to a larger data type is possible and is executed implicitly by the compiler:

Declaration	<pre>VAR VariableINT : INT; VariableREAL : REAL; END_VAR</pre>
Program code	<pre>VariableINT := 32767; VariableREAL := VariableINT + 100;</pre>

There are no problems when assigning an INT variable to a REAL variable, since REAL has a larger range of values.



Assigning a data type with a larger range of values to a smaller data type is not possible and is displayed with a corresponding error message:

Declaration	<pre>VAR VariableINT : INT; VariableREAL : REAL; END_VAR</pre>
Program code	<pre>VariableREAL := 7.0; VariableINT := VariableREAL / 2;</pre>

Error message

All

1 Error

4 Warnings

8 Messages

#	Category	Date/Time	Description	Error
10	Error	20.05.2025 10:08:41.0335	Data type mismatch: Cannot convert REAL to INT.	1140

When assigning a REAL variable to an INT variable, 32 bits must be reduced to 16 bits. This is not easily possible.



Implicit type conversions should be avoided if possible. Where necessary, any type conversion should be explicit in the code.
Explicit type conversions make the code easier to understand and maintain.

Explicit data type conversion

Explicit data type conversions are deliberately performed by the programmer using the corresponding function from the AslecCon library.
It must always be taken into account that type conversions can mean losing accuracy (e.g. omitting decimal places) or limiting/shifting a value range.



The error shown above can be prevented by using function `REAL_TO_INT()`.

Declaration

```
VAR
    VariableINT   : INT;
    VariableREAL  : REAL;
END_VAR
```

Program code

```
VariableREAL := 7.0;
VariableINT  := REAL_TO_INT(VariableREAL) / 2;
```

Result

The value 3 can be stored in VariableINT.
Explanation: The value of VariableREAL is "shortened" to INT (7.0 becomes 7). Then an integer division (7/2) is performed. The result (3) can be stored in VariableINT.

Sometimes an explicit type conversion can be used to get the maximum possible accuracy in a calculation.



Declaration

```
VAR
    VariableINT   : INT;
    VariableREAL  : REAL;
END_VAR
```

Program code

```
VariableINT := 7;
VariableREAL := INT_TO_REAL(VariableINT) / 2;
```

If no explicit type conversion were performed in the example shown, the result stored in VariableREAL would be 3.0, since an integer division would first be performed on the right side.



[Programming \ Libraries \ IEC 61131-3 functions \ AslecCon \ Function blocks and functions](#)

Exercise: Converting data types

Explicit data type conversion

- 1) Create one variable of type INT and one of type REAL.
- 2) Divide the REAL type variable (after assigning a value) by 2 and try to store the result in the INT type variable. What error message is generated?
- 3) Now use the correct conversion function for an explicit type conversion to prevent the error.

Consequences of type conversions

- 1) Find out about the behavior of the decimal places when converting a REAL variable into an INT value.
- 2) What happens if the value of the REAL variable exceeds the value range of the INT variable being converted to?

3.2 Derived data types

User-defined data types can be created that are based on the different base data types. This procedure is called derivation. User-defined data types consist of elements from the base data types.

User-defined data types, enumerations and derivations are always stored in files with the extension .typ. When adding a new program, a data type declaration is automatically created. If this file is not needed, it can be removed again.

Derived data types include:

- Direct derivatives and subranges
- Enumerations
- Structures
- Arrays and multidimensional arrays



[Programming \ Variables and data types \ Data types \ Derived data types](#)

(Multidimensional) arrays play an important role in programming and are therefore covered in a separate chapter ([4.1 "Arrays" on page 29](#)).

3.2.1 Direct derivatives and subranges

It is possible to derive basic data types directly. A new data type with a new name is created that has the same properties as the basic data type. An initial value can also be assigned to the new data type. All variables that use these data types therefore have the configured value.

In the case of direct derivation, a range of values can also be specified; this is then referred to as a subrange. Variables declared with this "new" data type can then only receive values that are in the specified range. Otherwise, an error message is generated informing that an invalid value assignment has been attempted. Variables can also be assigned a subrange.

	Data type with a subrange	<pre>TYPE TemperatureType : UINT(250..500); END_TYPE</pre>
	Variable with a subrange	<pre>VAR Temperature : UINT(200..550); END_VAR</pre>

Table 4: Declaration of a data type and a variable with a subrange



[Programming \ Variables and data types \ Data types \ Derived data types \ Direct derivatives](#)
[Programming \ Variables and data types \ Data types \ Derived data types \ SubrangesProgramming \ Variables and data types \ Data types \ Derived data types \ Subranges](#)



Attempting to assign a value outside the subrange to a variable whose value range is restricted by a subrange in the program is prevented by the compiler and an error message is generated. If the invalid assignment is made at runtime by another variable, the value is still written. Resulting mal-functions can be detected e.g. with IEC Check ([4.4.2 "IEC Check library" on page 43](#)).

Exercise: Declaring a direct derivative with a subrange

The data type "TemperatureType" should be derived directly. The base data type should be INT. The valid value range is between 200 and 500. Use numeric constants to define this value range.

- 1) Declaring a directly derived type
- Open the data type declaration, select symbol "Add directly derived type", then assign name "TemperatureType" and base data type "INT".
- 2) Declaring the constants
- Open the variable declaration, create constants MIN_TEMP and MAX_TEMP and assign the values.
- 3) Using the constants as a subrange of TemperatureType

Now use the two constants as the value range for the newly created data type.

4) Testing the result

Declare a new variable of data type `TemperatureType` in the variable declaration. Then try to assign a value outside the valid value range in the program code and view the compiler output in the message window.

5) Optional: Invalid assignment at runtime

Try assigning a value outside the allowed subrange at runtime using another variable.

3.2.2 Enumerations

An enumeration can be described as a list of values. This is also called "enumeration". After the declaration, the enumerated data type can be used as data type of a variable. Instead of the numerical value of the enumerated element, the variable contains the corresponding text. The numerical value of the enumeration element can still be read and written, however.

The values of the enumerated elements are numbered automatically in ascending order beginning with 0. Negative initial values can also be assigned.

Enumerated data types and their elements can be used for step sequences in PLC programming and are a way to make the source code more readable.

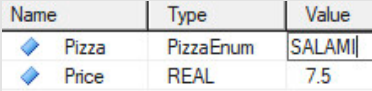
 Enumeration declaration	<pre> TYPE PizzaEnum: (MARGHERITA, SALAMI, FUNGHI, TUNA); END_TYPE </pre>	 Figure 13: Types.typ
Declaring the variables	<pre> VAR Pizza : PizzaEnum; Price : REAL; END_VAR </pre>	 Figure 14: Variables.var
Program code	<pre> CASE Pizza OF MARGHERITA: Price := 7.5; SALAMI: Price := 9; FUNGHI: Price := 8.5; TUNA: Price := 10.2; END_CASE </pre>	 Figure 15: Watch window

Table 5: Declaration of an enumeration and use in the program code

Enumerated types are managed in the background as DINT. Thus, a variable of the type of any user-defined enumeration always requires 4 bytes in memory.

Exercise: Enumerated data type "Pizza"

- 1)

Declare a global enumeration named `PizzaEnum` with at least three entries.
- 2)

Declare a variable of type `PizzaEnum` and a variable for the price of type `REAL`.
- 3)

Use both variables in a CASE statement in the cyclic section of the program to set the price per type of pizza.
- 4)

Observe the value of the price in the Watch window when a pizza is changed.

3.2.3 Structure types

A structure is composed of individual elements, those being basic data types, arrays or other structures. The entire structure is addressed via a common name. Each of the individual elements also has its own name.

Declaring structures is done either locally or globally in the corresponding .typ file, depending on the required scope.

Structures are primarily used to group together data and values that have a relationship to each other. An example would be a recipe that always uses the same ingredients, but in different amounts.

Default values for the individual structure elements can be set in the structure declaration. The default values can be overwritten in the variable declaration if necessary.

pizza::Types.typ [Data Type Declaration]		
Name	Type	Value
PizzaType		
Price	USINT	
Size	USINT	
Topping	PizzaEnum	
LeaveCheese	BOOL	
BakingTemperature	TemperatureType	430

Figure 16: Declaration of a structure in Automation Studio

	Structure declaration	<pre>TYPE PizzaType : STRUCT Price : REAL; Size : USINT; Topping : PizzaEnum; LeaveCheese : BOOL; BakingTemperature : TemperatureType := 430; END_STRUCT; END_TYPE</pre>
	Variable declaration	<pre>VAR Pizza : PizzaType; END_VAR</pre>
	Use in the program code	<pre>// Access to elements of Pizza Pizza.Topping := FUNGHI; Pizza.Price := 8.5; Pizza.BakingTemperature := 400;</pre>

Table 6: Declaration of a structure and use in the program code



[Programming \ Variables and data types \ Data types \ Derived data types \ Structures](#)
[Programming \ Editors \ Table editors \ Declaration editors \ Table editor for data type declaration](#)

Exercise: Declaring structure "PizzaType"

Declare a global structure named `PizzaType`.

This structure should include the following elements:

- Price (REAL)
- Size (USINT)
- Topping (PizzaEnum)
- LeaveCheese (BOOL)
- BakingTemperature (TemperatureType)

1) Structure declaration

Assign the name `PizzaType` in the declaration editor. Add the elements with the corresponding data types according to the list provided.

2) Setting default values

Set the `BakingTemperature` in the declaration of the structure to the default value 430 for all variables.

3) Initializing the elements in the program code

Declare a new variable named `Pizza` and apply the new data type. Use the variable in the program code and initialize the elements with values. Then view the result in the Watch window.

Nested structures

The elements of a structure can also be a structure data type. This is also referred to as structural nesting. In practice, nesting depths of up to four levels are common. This improves the structuring and readability of the source code by better representing the dependencies or hierarchy of otherwise individual variables.



The above example is now adapted so that the element `Price` of the structure `PizzaType` is no longer a simple REAL, but a separate structure variable with the elements `Euro` and `USD`.

Structure declaration

To declare a nested structure, a structure data type declared in the project is used for the desired structure element instead of a base data type.

The order of the structure declarations `PriceType` and `PizzaType` is not important, but attention should be paid to the scope of `PriceType`.

TYPE

```
PriceType : STRUCT
  Euro : REAL;
  Usd : REAL;
END_STRUCT;
PizzaType : STRUCT
  Price : PriceType;
  Size : USINT;
  Topping : PizzaEnum;
  LeaveCheese : BOOL;
  BakingTemperature : TemperatureType := 430;
END_STRUCT;
END_TYPE
```

Variable declaration

The declaration for the structure variable `Pizza` does not change.

VAR

```
Pizza : PizzaType;
ExchangeRate : REAL := 1.11;
END_VAR
```

Use in the program code

The individual elements are described or read in the case of nested structures via repeated use of the "." operator.

```
Pizza.Price.Euro := 8.5;
Pizza.Price.Usd := Pizza.Price.Euro * ExchangeRate;
```

Table 7: Declaration of a nested structure and use in the program code

Task: Nested structure

Modify the existing structure `PizzaType` so that the price for a pizza can be specified in three different currencies.

- 1) Declare the structure `PriceType` with three elements for different currencies (e.g. Euro, USD, Forint).
- 2) Adjust the data type of the `Price` element of `PizzaType`.
- 3) Create additional variables for the exchange rates.
- 4) Set the euro price of `Pizza` in the program and calculate the prices in the other currencies using the exchange rates.
- 5) Observe the change in value of the structure elements for the different currencies in the Watch window when changing the exchange rates or the euro price.

3.3 Memory requirements of variables

Each variable or constant uses a certain amount of memory at runtime. The required amount of memory depends on the data type and partly also on the order in which the variables are stored in memory.

sizeof for determining the memory requirements

The memory requirement of a variable can be determined with the function `sizeof()`.

The function receives the name of the variable as parameter and returns the used memory in bytes as return value.

```
// DECLARATIONS
VAR
    Pizza : PizzaType;
    Length : UDINT;
END_VAR

// CYCLIC
Length := sizeof(Pizza);
```



[Programming \ Libraries \ IEC 61131-3 functions \ OPERATOR \ Functions \ Address and Size Functions \ sizeof\(\)](#)

For the configuration of an error-free application it is necessary to have an understanding of how variables can be stored and localized in memory.

3.3.1 Variable addresses

Access to the actual location in the hardware is based on the address.

The addresses of variables and constants are unchangeable at runtime of the system. These variables are therefore referred to as "static". A change to the address of a static variable happens when the controller is restarted or when processes and data are reloaded into the DRAM during transfer.

The addresses of remanent and permanent variables also change when the controller is restarted, since the data is stored during restart not in DRAM but in the battery-backed SRAM (or FRAM).

Determining the addresses of variables**Declaring the variables**

```
VAR
    Variable1 : INT := 123;
    Variable2 : INT := 77;
    Address1 : UDINT;
    Address2 : UDINT;
END_VAR
```

Determining the addresses

```
Address1 := ADR(Variable1);
Address2 := ADR(Variable2);
```

In ST, the address of a variable can be determined with the function `ADR()`. Since an address is a 32-bit binary number, the return value of `ADR()` corresponds to a `UDINT`.

Therefore, the address variables in the example (`Address1` and `Address2`) have the data type `UDINT`.

For data types that require more than one byte of memory (e.g. INT), the corresponding amount of consecutive memory cells is used.

In the case of `Variable1`, the system knows that it must always read (or write) two consecutive cells because of the declaration of the data type (INT).

In the example, both `Variable1` and `Variable2` use 2 bytes (i.e. 16 bits) of memory each, which corresponds to two memory cells each. The variables `Address1` and `Address2` use 4 memory cells each, since the data type UDINT requires 32 bits (i.e. 4 bytes).

address	memory cell	name of variable
0x0471DC7E		
0x0471DC7C		
0x0471DC7A	0x0471DC72 (address of Variable2)	Address2
0x0471DC78		
0x0471DC76	0x0471DC70 (address of Variable1)	Address1
0x0471DC74		
0x0471DC72	77	Variable2 (type: INT)
0x0471DC70	123	Variable1 (type: INT)

Figure 17: Addresses and memory requirements of variables



A memory address can be identified in the program code but is not permitted to be used as a fixed value because a new address can already be assigned by Automation Runtime during the next controller startup or project transfer.



If new variables are created in the project or existing ones are deleted, address changes occur after a transfer of the project even without a restart. Therefore, a restart is recommended after such a transfer.



[Programming \ Libraries \ IEC 61131-3 functions \ OPERATOR \ Functions \ Address and Size Functions \ ADR\(\)](#)

3.3.2
Alignment

Different process architectures follow different rules when it comes to data storage in memory. In very few cases, the user has to take this into account. If data is to be transferred between systems that have different architectures, however, it is a good idea to think about how that data is to be stored.

The arrangement of variables in memory is called alignment. Depending on the system, the data type used and the order in which the variables are declared, there may be pad bytes, i.e. unusable memory cells.

Pad bytes

`Variable1` is declared with type SINT in this example, which requires only 1 byte of memory according to IEC.

Although a variable of the type SINT uses only 1 byte of memory according to IEC, it turns out that the next used memory cell is only two addresses further.

The reason for this is that the system can only assign addresses to variables that are divisible by the memory requirements of the data type (two bytes in the example). This can result in memory cells that remain unused. These are called pad bytes.

If data types with less than 2 bytes of memory are combined in structures or arrays, odd memory cells are also used for the individual elements that follow one another in memory.

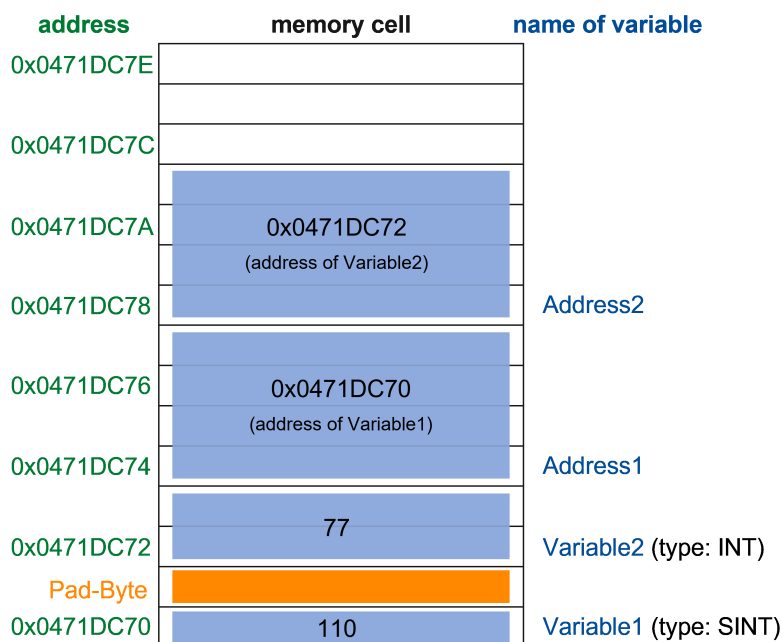


Figure 18: Pad bytes between variables of different data type

Exercise: Determining the addresses of variables

- 1) Identify the addresses of the variables existing in the program. To do this, create address variables of type UDINT to record the addresses and then display their values in the Watch window.
- 2) Change the declaration order of the variables. Does this have an effect on the addresses of the variables?
- 3) Restart the controller and view the addresses and the order of the variables again. What can be seen?
- 4) Now try to use the findings and the declaration of a variable with a memory requirement of only one byte to generate a pad byte in memory.

Alignment of structural elements

When declaring structures, the rules of alignment also apply to the individual structure elements. This can lead to pad bytes between the individual structural elements.

The declaration order of the elements of a structure thus has an influence on how much memory a variable of this data type uses in the system.

Declaration order without pad bytes

```

TYPE
  PizzaType : STRUCT
    Price : REAL;
    Topping : PizzaEnum;
    Size : USINT;
    LeaveCheese : BOOL;
    BakingTemperature : TemperatureType;
  END_STRUCT;
END_TYPE

```

Declaration order with pad bytes

```

TYPE
  PizzaType : STRUCT
    Size : USINT;
    Price : REAL;
    LeaveCheese : BOOL;
    Topping : PizzaEnum;
    BakingTemperature : TemperatureType;
  END_STRUCT;
END_TYPE

```

Table 8: Effects of the declaration order on the memory requirements of a structure variable

Declaration order without pad bytes

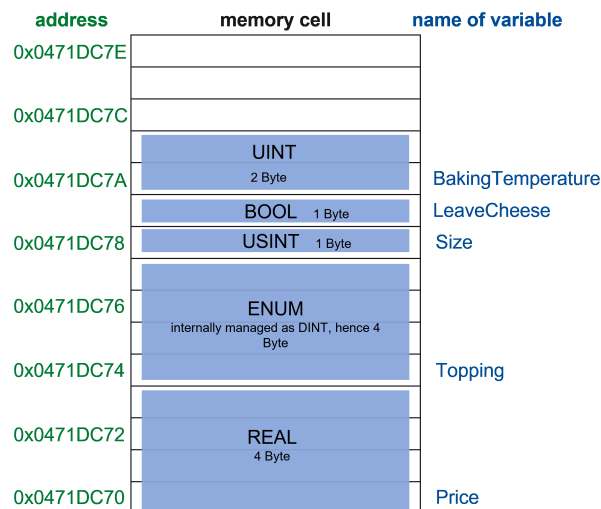


Figure 19: Declaration order without pad bytes

Declaration order with pad bytes

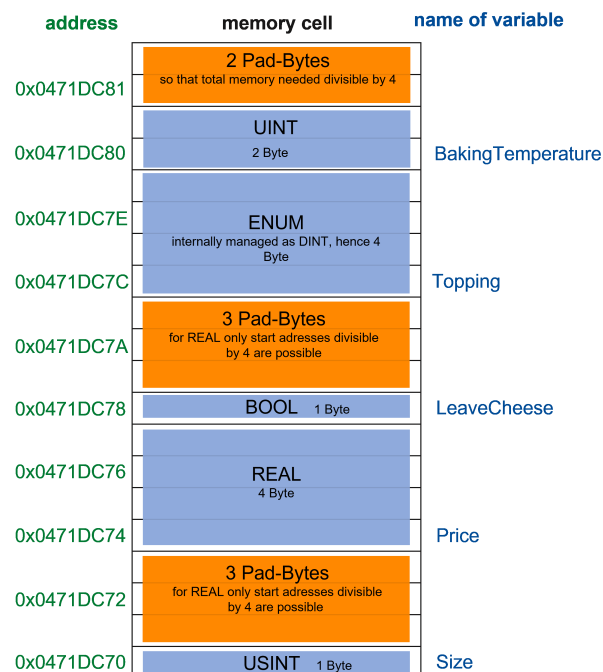


Figure 20: Declaration order with pad bytes

Table 8: Effects of the declaration order on the memory requirements of a structure variable


[Simulation \ ARsim \ Runtime behavior \ Alignment](#)

Exercise: Memory requirement of PizzaType

- 1) Use the function `sizeof()` to find out how much memory a variable of type `PizzaType` with the declaration from the previous exercise needs.
- 2) Consider between which elements the pad bytes, if any, are located in the structure and how they could be re-arranged to make them disappear.

3.4 Dynamic memory access (pointers)

All variables shown so far were static variables. For static variables, the correlation between the address (memory cell) and the variable name cannot be changed at runtime. This means that the name of a variable is always the identifier for a specific memory cell (for the entire runtime duration).

In contrast, there are also variables that are not always linked to the same address (and therefore the same memory cell) at runtime, however. These are reference variables. Reference variables, references for short, are often also called pointers.¹³

¹³ In programming, there are, in fact, a few small differences between pointers and references. They can be considered identical, however, for general functionality and the memory view.

The terms pointer and reference are used synonymously in the following sections. References are described in the "Dynamic variables" section of Automation Help.¹⁴

A pointer (or reference) is itself also a variable, but of the type REFERENCE TO. While variables as they are normally know store values (e.g. an integer), a pointer stores an address. The value of a pointer is thus an address. This address refers to the memory cell that the pointer is "pointing to" (referencing). **A pointer can be used to read or overwrite the value of the memory cell being referenced.** This is also possible if there is simultaneously a link between this memory cell and a static variable name.

3.4.1 Declaring and using pointers

Declaration

Pointers (references) are marked in ST with the keyword REFERENCE TO during the declaration. The data type to which the reference (the pointer) is to refer later must already be specified during the declaration. It is not possible to initialize a reference during the declaration; this can only be done at runtime.

At runtime, a pointer can refer to changing addresses and therefore different variables. A pointer has access to a specific address (and therefore a specific memory cell) via the keyword ACCESS, which is followed by the address of the variable to be accessed.



The example shows the declaration and use of three static variables and two pointers.

Declaring the variables

```
VAR
  Variable1 : INT := 123;
  Variable2 : INT := 77;
  Variable3 : REAL := 12.3;
  Pointer1 : REFERENCE TO INT;
  Pointer2 : REFERENCE TO REAL;
END_VAR
```

Program code

```
Pointer1 ACCESS ADR(Variable1);
Pointer1 := 33;
Variable2 := Variable2 + Pointer1;

// Move pointer to another memory cell
Pointer1 ACCESS ADR(Variable2);
```

Instead of the identifier Variable1, Pointer1 can now also be used in the program. For both read and write accesses, the memory cell currently pointed to by Pointer1 is accessed directly.

¹⁴ From a programming point of view, a reference is not a dynamic variable. For a real dynamic variable, the memory required by the variable would be requested dynamically and reserved by the system at runtime.

View in memory

A pointer references a static variable if its address is specified after the keyword ACCESS.

```
Pointer1 ACCESS ADR(Variable1);
```

This is also termed "Pointer1 points to Variable1".

The pointer itself only "remembers" the address of the variable it is supposed to point to. The pointer provides access to the contents of the referenced memory cell. In ST, the normal assignment operator "!=" can be used for this purpose; no dereferencing operator is needed as it is the case in ANSI C.

```
Pointer1 := 33;
```

This assigns the new value 33 to Variable1 because Pointer1 currently points to Variable1.

A pointer can also be used to read the value of the variable (content of the memory cell) that is being referenced:

```
Variable2 := Variable2 + Pointer1;
```

The value of the memory cell to which Pointer1 currently points is read. In the example, 33 is thus read and added to variable2.

A pointer can also be "moved" to another memory cell at runtime:

```
Pointer1 ACCESS ADR(Variable2);
```

This makes it possible to access different locations in memory with a pointer at different locations in the program code.

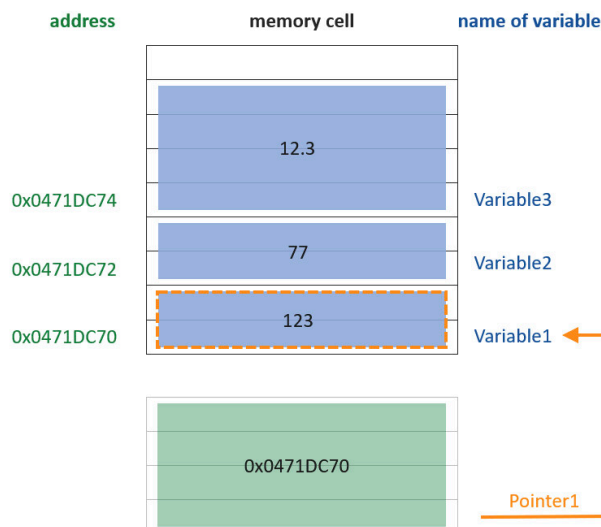


Figure 21: Reference of Pointer1 to Variable1

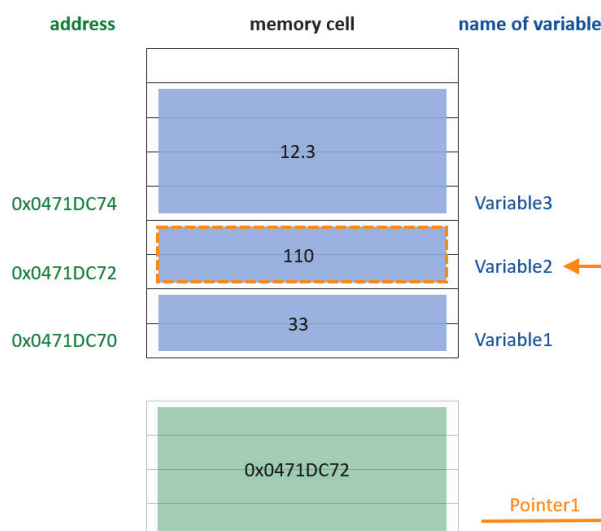


Figure 22: Reference of Pointer1 to Variable2



[Programming \ Variables and data types \ Variables \ Dynamic variables](#)

3.4.2 Significance of data types for pointers

A pointer actually "stores" an address, namely the address of the variable it is referencing. An address is 32 bits long, so a pointer requires 32 bits in memory, which corresponds to a UDINT¹⁵.

A data type must nevertheless be specified when declaring a pointer. The reason is that only the start address of a variable is retrieved with ADR(). The size of the data type then determines how many subsequent bytes are read in the memory.

¹⁵ Addresses are 32 bits (i.e. 4 bytes) long, never negative and always an integer. Of the 4-byte IEC data types (DINT, UDINT, REAL), only UDINT has all those properties.

With a pointer to INT, for example, only two memory cells need to be used, while with a pointer to REAL, four memory cells belong to the variable and must therefore be read or written.

If the data type of the pointer does not correspond to the data type of the variable, the memory areas are interpreted incorrectly and may be overwritten if four bytes are written instead of two bytes, for example.

```
Pointer1 ACCESS ADR(Variable3);
Pointer1 := 100;
```

With this invalid access due to the mismatching data type of the pointer (INT) and the variable (REAL), the statement `Pointer1 := 100;` will not have the desired effect.

The value of `Variable3` does not change to 100, and instead it can be observed in the Watch window that 12.3 is overwritten by 12.250004. The reason for this is that `Pointer1` (due to its declaration `REFERENCE TO INT`) can only manipulate the lower two memory cells of `Variable3`.

```
Pointer2 ACCESS ADR(Variable3);
```

Only a pointer to REAL can read and store the information correctly. Even an integer data type with 4 bytes (e.g. `REFERENCE TO UDINT`) will not deliver a correct result here, because the interpretation of the binary data in memory is different for the two data types (REAL and UDINT)!

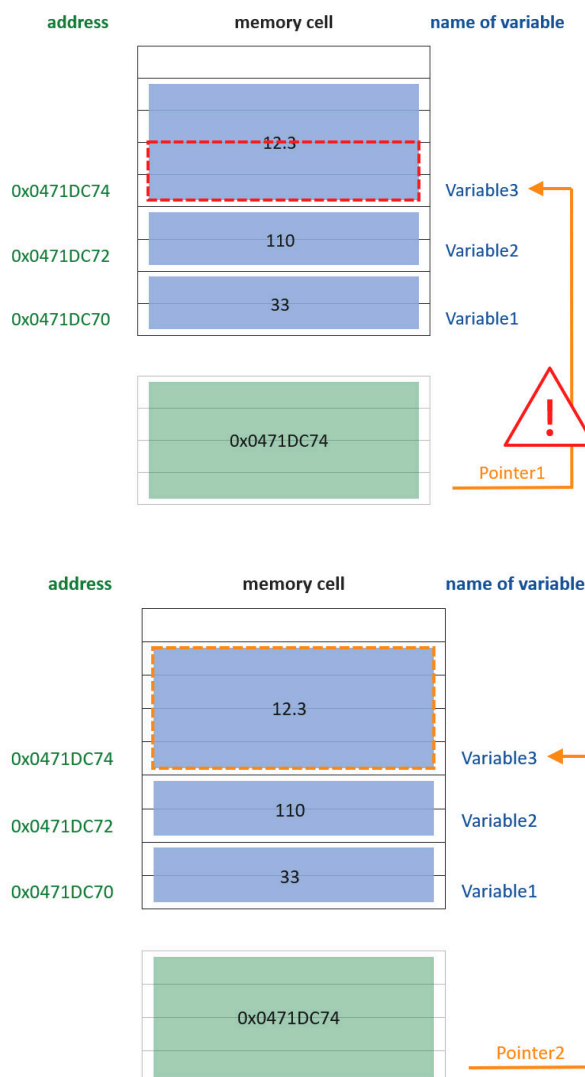


Figure 23: Significance of the data type for pointers

This concept becomes especially important with a pointer to a structure variable. Due to the type declaration of the structure, the compiler knows the organization of the data in memory belonging to this structure and can take this into account accordingly when accessing it by means of a pointer. `ADR()` also only provides the **start address** for a structure variable in memory **from which point** all elements belonging to the structure variable are stored.

Errors in programming can cause the start or end addresses of the read or write operation of memory cells that are actually belonging together to be selected incorrectly. Then data is read or written incorrectly.

This can lead to very hard to find and sporadic errors, e.g. if this unintentionally overwrites the value of another variable. Certain memory addresses are protected by the system so that writing or reading will cause a system crash (EXCEPTION Page fault), thus allowing the error to be detected. This is described in more detail in the chapter 4.4 "Error handling with IEC Check" on page 41.

Optional task: Pointer to sub-area of an array

Three elements of an array with a total of 10 elements (`BaseArray1`) of type `USINT` should be read using a pointer (`PointerArray`) in response to a start command (`ReadSelection`). The position of the three elements to read can be specified via `SelectedIndex`.

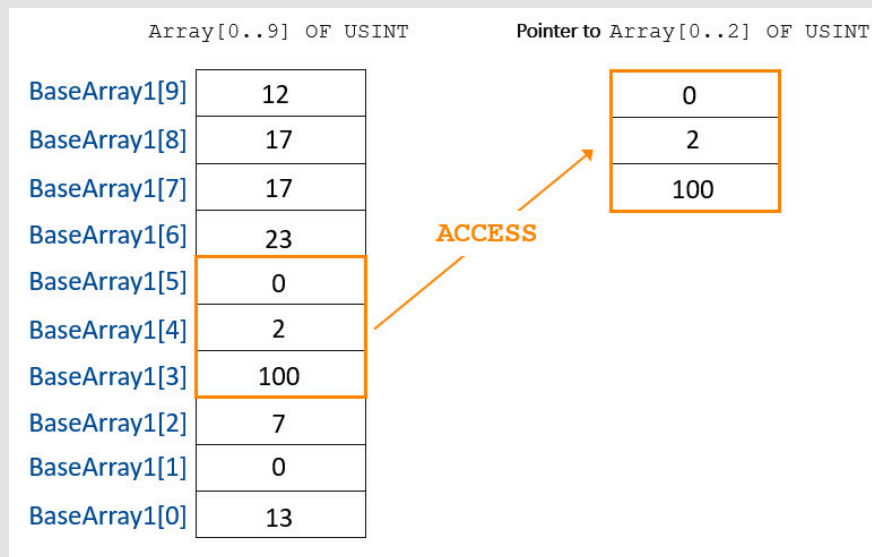


Figure 24: Schematic representation of the reading process with pointers

- 1) Create an array with 10 elements of type USINT and create a pointer to a three-element array of type USINT.
- 2) Create the program code that executes the read process using the pointer after setting start command `ReadSelection`. It should be possible to change the index from which the three values of `BaseArray1` are read at runtime via variable `SelectedIndex`.
- 3) Now try creating a second array with 10 elements (`BaseArray2`), which directly follows `BaseArray1` in memory in order to observe a read limit violation if you set `SelectedIndex` to 9.



Dynamic memory access and the use of pointers are particularly important when using arrays or transferring large data structures to a function or function block.

4 Arrays and strings

While arrays also belong to the derived data types, a string is a base data type according to IEC 61131-3. Because of their identical management in memory and the resulting identical risks in use, they are discussed together here.

4.1 Arrays

Unlike structures, arrays can only contain elements of the same data type. When declaring the array, the common data type of the elements is specified. The individual elements are not addressed via their own element names, as is the case with structures, but via the common identifier (array name) and their respective index.

Arrays can also be called fields.

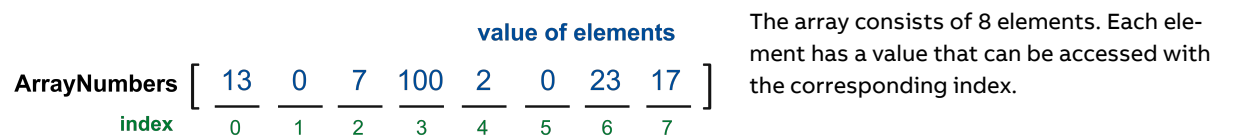


Figure 25: Schematics of an array

4.1.1 Declaring and using arrays

Declaration

Arrays are declared in the .var file via the keyword `ARRAY`. The number of elements and the data type must be specified in the declaration. The number of elements is also called the size of the array.

	Declaration	<pre>VAR ArrayNumbers : ARRAY[0..7] OF USINT; Value : USINT; index : SINT; END_VAR</pre>
	Program code	<pre>// Assign values to single elements of array ArrayNumbers[0] := 13; ArrayNumbers[1] := 0; ArrayNumbers[6] := 23; // Read a value from array Value := ArrayNumbers[6];</pre>

Table 9: Declaration of an array with 8 elements of type USINT

Individual elements of the array are accessed via the respective index, which is enclosed in square brackets after the name. Both read and write access is possible.

The index used can be (as in the example) a fixed number, a variable, a constant or an enumerated element. Especially when using variables as indices, care must be taken that only the valid indices according to the array declaration are used; otherwise memory access violations may occur. In the example, this would be all numbers from 0-7.

Array in memory

In memory, the elements of an array are stored in sequence. There are no pad bytes between the elements if the data type of an element (as in the example USINT) requires only one byte of memory.

In principle, the access to the individual elements can be imagined as "continuous" variable names.

`ArrayNumbers[2]` is thus the identifier for the memory cell with address `0x0471DC72` and the value 7.

`ADR(ArrayNumbers[2])` would return the address `0x0471DC72` accordingly.

address	memory cell	name of variables
	255	
0x0471DC78		
	17	ArrayNumbers[7]
0x0471DC76	23	ArrayNumbers[6]
	0	ArrayNumbers[5]
0x0471DC74	2	ArrayNumbers[4]
	100	ArrayNumbers[3]
0x0471DC72	7	ArrayNumbers[2]
	0	ArrayNumbers[1]
0x0471DC70	13	ArrayNumbers[0]

Figure 26: Storage of an array in memory

With `ADR(ArrayNumbers[0])` or `ADR(ArrayNumbers)`, you would get `0x0471DC70`.



This shows what happens if the index used is outside the valid range:

```
ArrayNumbers[9] := 255;
```

This statement attempts to execute an access to a memory cell (with address `0x0471DC79`) that no longer belongs to the array. If the invalid index is a fixed number in the source code, this is already addressed with an error message during compiling. However, if the invalid access occurs only at runtime through a variable used as an index, a memory access violation occurs that may result in a system error (see [4.4.1 "Causes of errors" on page 41](#)).

Using arrays

Arrays are often used to collect data that requires identical actions or calculations to be performed.

In this case, the index is not a fixed number, but a variable that can be used to loop "over the array" (iterate). Here, too, both read and write access is possible.

```
FOR index := 0 TO 7 DO
  ArrayNumbers[index] := index * 2;
END_FOR
```

Each of the 8 elements (index 0-7) is assigned twice the value of its own index. The last element thus has the value 14.

Arrays with structure elements

Array elements can also be structures with respect to data type. They are used in the same way as arrays with elements of basic data types. The access to single elements of the array is still done with the index, the access to the elements of the structure behind it with a dot.



Declaration

```
VAR
  PizzaList : ARRAY[0..9] OF PizzaType;
END_VAR
```

Program code

```
PizzaList[0].Price.Euro := 7.5;
```



References as elements of arrays are not possible in Automation Studio and generate an error message during compiling.



It is important to ensure that array elements are only accessed with indices that are in the valid index range of the declared array!

4.1.2 Range specifications with constants

Since using fixed numerical values in the declarations and program code usually leads to programming that is unmanageable and difficult to maintain, the use of numerical constants is recommended.

The upper index of the array can be specified by a constant. The same constant can be used in the program code for limiting the index variable (`index`) in loops.



Declaration

```
VAR CONSTANT
    MAX_INDEX : USINT := 99;
END_VAR
VAR
    ArrayREAL : ARRAY[0..MAX_INDEX] OF REAL;
    index : USINT;
END_VAR
```

Program code

```
FOR index := 0 TO MAX_INDEX DO
    // Operation on element of array
    ArrayREAL[index] := index * 5.0;
END_FOR
```

With this program code, the size of the array can be adjusted by changing the value of the constants only when they are declared. The rest of the code works unchanged.



[Programming \ Variables and data types \ Data types \ Derived data types \ Arrays](#)

Correlation between MAX_INDEX and number of elements

As already seen when declaring arrays, MAX_INDEX must always be one less than the number of elements in an array, since the first element of the array has the index 0.

In principle, you can also find the size of an array via the source code. To do this, `SIZEOF()` is used to determine the total size (i.e. memory requirement) of the array, which is then divided by the size of any element in the array. This provides the number of elements in the array.

```
NumberOfElements := SIZEOF(ArrayREAL) / SIZEOF(ArrayReal[0]);
```

As an alternative to the example above, which uses MAX_INDEX to limit the loop, the following can also be done:

```
FOR index := 0 TO SIZEOF(ArrayREAL) / SIZEOF(ArrayREAL[0]) - 1 DO
    // Operation on element of array
    ArrayREAL[index] := index * 5.0;
END_FOR
```

This variant of a loop also remains valid without change when the size of ArrayREAL in the declaration file is changed and does not require any adaptation of the source code in the cyclic section of the program.

4.1.3 Initial values of arrays

In some cases it is necessary or desirable to fill an array with values already during declaration. This can be done in the variable editor in both text and table view.

Filling with different initial values



```
VAR
    ArrayNumbers : ARRAY[0..7] OF SINT := [13,0,7,100,2,0,23,17];
END_VAR
```

Name	Type	Value
ArrayNumbers	SINT[0..7]	
ArrayNumbers[0]	SINT	13
ArrayNumbers[1]	SINT	0
ArrayNumbers[2]	SINT	7
ArrayNumbers[3]	SINT	100
ArrayNumbers[4]	SINT	2
ArrayNumbers[5]	SINT	0
ArrayNumbers[6]	SINT	23
ArrayNumbers[7]	SINT	17

The known `ArrayNumbers` with 8 elements of type `SINT` is declared here and initialized with the values shown in the example.

This method is only useful and manageable for arrays of small size. Test arrays are often created in this way during program development.

Filling with identical initial values for all array elements

If all elements of an array should be initialized with the same initial value, there is another way to specify this when declaring the array.



```
VAR
    ArrayREAL : ARRAY[0..99] OF REAL := [100(12.0)];
END_VAR
```

Name	Type	Value
ArrayREAL	REAL[0..9]	
ArrayREAL[0]	REAL	12.0
ArrayREAL[1]	REAL	12.0
ArrayREAL[2]	REAL	12.0
ArrayREAL[3]	REAL	12.0
ArrayREAL[4]	REAL	12.0
ArrayREAL[5]	REAL	12.0
ArrayREAL[6]	REAL	12.0
ArrayREAL[7]	REAL	12.0
ArrayREAL[8]	REAL	12.0
ArrayREAL[9]	REAL	12.0
ArrayREAL[10]	REAL	12.0
ArrayREAL[11]	REAL	12.0
ArrayREAL[12]	REAL	12.0

In the example, 100 `ArrayREAL` elements are initialized with the value 12.0 using this type of initialization. Since the array has only 100 elements, all of them are initialized.

With the specification

```
[50(12.0), 20(24.0), 30(230.0)]
```

the first 50 elements of the array would be initialized with the value 12.0, the next 20 elements with the value 24.0 and the last 30 elements of the array with the value 230.0.

Exercise: Declaring and using an array

Declare an array with elements of type `PizzaType`. The array should contain 10 elements. Use a constant instead of a fixed value when declaring the array. Calculate the average price of a pizza. Declaring and initializing

- 1) Declare a variable `PizzaList` as an array with 10 elements of type `PizzaType`.
- 2) Initialize all array elements by default with `Size 32` and the `Topping MARGHERITA`.
- 3) Manually assign values to the price of all array elements in the `INIT` part of the program or in the Watch window. Calculating the average value of "Price"
 - 1) Go through all array elements and form the sum of the `Price.Euro` or `Price.Usd` elements.
 - 2) From this, calculate the average price of a pizza.

Test if the average price changes by changing the `Price` element of the first and last array elements. This is an easy way to make sure that the program is working properly.

Use the constant used in the array declaration for both the summation and the averaging.

Exercise: Using a pointer to an array element

Individual elements of an array can be accessed using a pointer. This is frequently used, especially when transferring arrays to functions or function blocks.

The value of an array element should be read using a pointer. At runtime, which element of the array is read can be selected by adjusting an index variable `IndexToRead`.

- 1) Use array `PizzaList` from the previous task or alternatively, create an array with 10 elements of type `REAL`. Initialize all elements with different values. Declare a pointer to the data type that the elements of your array have.
- 2) In the cyclical part of the program, ensure that the pointer reads the element of the array specified via `IndexToRead` if a `ReadElement` trigger command has been set.
- 3) Test your program by changing the value of `IndexToRead` in the Watch window and observing the value of the pointer.

4.1.4 Multidimensional arrays

Arrays can also be composed of several dimensions. Accessing an element of the array is then no longer done via a single index, but via as many indices as the array has dimensions.

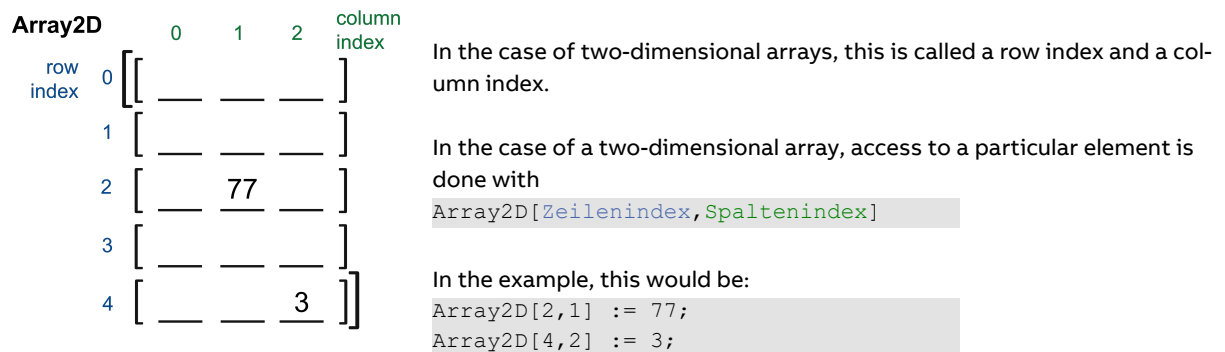


Figure 27: Schematics of a two-dimensional array

The use of constants to set array limits is also recommended for multidimensional arrays, both when declaring and when using index limits in loops.

 Declaration	<pre> VAR CONSTANT MAX_INDEX_ROW : USINT := 4; MAX_INDEX_COLUMN : USINT := 2; END_VAR VAR Array2D : ARRAY[0..MAX_INDEX_ROW,0..MAX_INDEX_COLUMN] OF INT; i : USINT; // Row index j : USINT; // Column index END_VAR </pre>
Program code	<pre> // Nested loop to reach all elements of array // with two indices i and j FOR i := 0 TO MAX_INDEX_ROW DO FOR j := 0 TO MAX_INDEX_COLUMN DO // Do here what ist necessary for each element of the array // Access with Array2D[i,j] END_FOR END_FOR </pre>

Optional exercise: Loading a pizza oven

The loading of a pizza oven should be represented by a two-dimensional array. The oven has 4 drawers; each drawer can hold a maximum of 10 pizzas at the same time.

- 1) Declare two constants with meaningful identifiers for the maximum number of drawers and the maximum quantity of pizzas per drawer.
- 2) Using the constants, declare a two-dimensional array of type BOOL named `OvenUsage`.
- 3) Assign the value "TRUE" to individual, random elements during declaration.
- 4) Calculate how many pizzas are currently in the oven in the cyclic program section and store the result in a suitable variable.
- 5) Check the functionality of the program by changing the values of individual elements of `OvenUsage` in the Watch window and by observing the result variable.

4.2 Strings

The arrangement of the individual elements of a string in memory is identical to that of arrays, but strings belong to the base data types according to IEC 61131-3.

4.2.1 Declaring strings

Arrays allow free choice of data type for each element, which is not possible with strings. The elements of a `STRING` always have a memory requirement of one byte, and the elements of a `WSTRING` have a memory requirement of two bytes.

The information stored in binary form in strings is interpreted either by the ASCII character map if it is a `STRING`, or by `UNICODE` if it is a `WSTRING`.

As with other data types, the declaration is made in the `.var` file via the name of the data type (`STRING` or `WSTRING`) followed by the required number of characters in square brackets.

```

VAR
    StringASCII : STRING[9] := 'Hallo!';
    StringUNICODE : WSTRING[9] := "Hallo";
END_VAR

```

Assigning a value to a `STRING` is done with single quotes, to a `WSTRING` with double quotes.

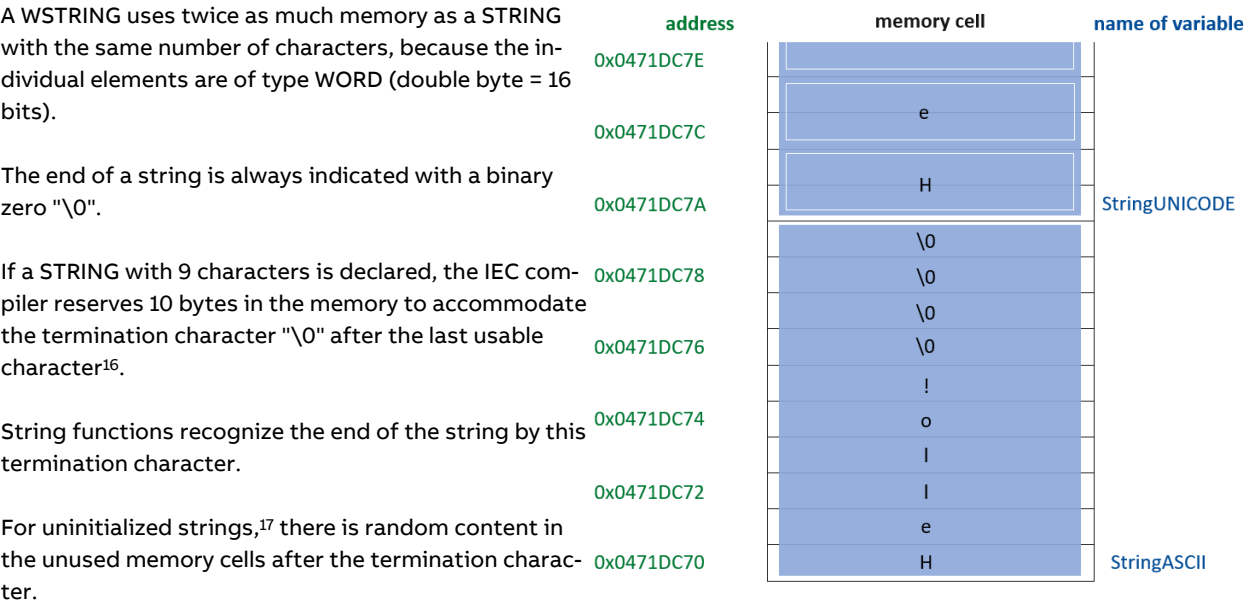


Figure 28: STRING and WSTRING in memory

Individual elements of a string cannot be addressed directly in ST as is the case with arrays. There are various string functions in different libraries for manipulating strings.

As with arrays, there is a risk of writing beyond the actual end of the string, which can also lead to memory access violations.

4.2.2 String length

In many cases it is necessary to find out how long a string is. However, a distinction must be made between

- the total amount of memory reserved for the string in memory and
- the number of characters (currently) used in this string.

Total amount of memory

The total amount of memory reserved for a string in memory is unchangeable at runtime of the program. Memory size can be determined using the function `sizeof()`.

`sizeof(StringASCII)`

returns the value 10 for the STRING from the above declaration (9 usable characters + "\0").

`sizeof(StringUnicode)`

returns the value 20 for the WSTRING from the above declaration (9 usable characters of 2 bytes each + "\0" with another 2 bytes).

The function `sizeof()` is given the name of the string to be examined as a parameter. The return value is the number of bytes used by the string.

The function `sizeof()` can be used for strings of type STRING as well as WSTRING.

¹⁶ For non-IEC languages (e.g. C), the string is only as large as specified in the declaration for the static string variable.

¹⁷ If a string is created in a .var file, it is initialized according to the IEC standard. Uninitialized strings can therefore only occur with dynamically allocated memory.

Number of characters used

To determine the length of a string, i.e. the number of characters actually used, different functions are required for the two types of string (STRING and WSTRING). The length of a string can change at runtime if the number of characters used changes.

Function	Library	Description
<code>LEN()</code>	Standard	Determines the number of characters used of a STRING specified by name.
<code>brsstrlen()</code>	AsBrStr	Determines the number of characters used of a STRING specified by address.
<code>brwcslen()</code>	AsBrWStr	Determines the number of characters used of a WSTRING specified by address.

Table 10: Comparison of string length functions

For the two strings declared above, the function calls look like this:

Function call	Description
<code>LEN(StringASCII)</code>	Returns the value 6 because the STRING was initialized with 6 characters.
<code>brsstrlen(ADR(StringASCII))</code>	Returns the value 6 like LEN().
<code>brwcslen(ADR(StringUNICODE))</code>	Returns the value 5 because the WSTRING was initialized with 5 characters.

Table 11: Examples of string length functions



The longer a string is, the longer the execution time of these functions, which in the worst case can lead to a cycle time violation. In a loop, the functions check the current byte for "\0".



For all functions that require an address as a parameter (here: `brsstrlen()`, `brwcslen()`), ensure that a null pointer is not transferred because this leads to a page fault.

Example: Determining string lengths

Comparison of STRING and WSTRING

1) Declaration

Declare a variable of type STRING and WSTRING with 80 characters each. Initialize both strings with different values. The identifiers should be `DescriptionSTRING` and `DescriptionWSTRING`.

2) Determining the string length

Use the appropriate functions to determine the length of both strings in the cyclic section of the program, i.e. how many characters of the string are actually used. (Note: `LEN()` or `brsstrlen()` for STRING, `brwcslen()` for WSTRING)

In addition, use `SIZEOF()` to determine the amount of memory occupied by both strings.

3) Extending the `PizzaType` structure variable by a STRING element

Add an element of type STRING with 100 characters called `Description` to the declaration of `PizzaType`. Initialize either all or a single element of the array `PizzaList` with any string and then try to determine the length of the string in the cyclic program.

4.2.3 String functions

While in ST it is possible to assign strings or compare two strings using the usual assignment or comparison operator, other programming languages require libraries to handle strings.



Declaration

```
VAR
  String1 : STRING[40] := 'Hello ';
  String2 : STRING[40] := 'World!';
END_VAR
```

Program code

```
IF String1 <> String2 THEN
  String2 := 'Hello World!';
END_IF
```

Library functions are also necessary in ST for the advanced handling of strings. The following section provides an overview of the available libraries and their areas of application. This makes it possible to concatenate strings, search for substrings in a longer string, replace text or insert strings at any position in other strings. Converting to numerical values is also possible.

Library	String type	Description
Standard	STRING	Standard functions for handling STRINGs, not available in ANSI C, length limitation for strings
AslecCon	(W)STRING	Conversion functions from and to (W)STRING, not available in ANSI C
AsBrStr	(W)STRING	Conversion and comparison functions (STRING only) Operations on memory areas: Compare, copy, set values (type-independent)
AsBrWStr	WSTRING	Standard functions for handling WSTRINGs



[Programming \ Libraries \ IEC 61131-3 functions \ STANDARD \ Function blocks and functions](#)
[Programming \ Libraries \ IEC 61131-3 functions \ AslecCon \ Function blocks and functions](#)
[Programming \ Libraries \ Configuration, system Information, runtime control \ AsBrStr](#)
[Programming \ Libraries \ Configuration, system information, runtime control \ AsBrWStr](#)

Important functions for handling strings

The following functions for handling strings from the AsBrStr library are often needed when working with strings. For further information and a complete list of available functions, see Automation Help. In particular, the STANDARD library contains functions for more complex handling of strings.¹⁸

The most common use cases are copying information from a source string (SourceString) to a destination string (DestString) or appending information to the existing content. Comparing two strings is also a very common use case.

```
VAR
  DestString : STRING[100];
  SourceString : STRING[100];
  NextAddress : UDINT;
  Result : DINT;
END_VAR
```

The following table provides a short overview of the three functions available for these cases from the AsBrStr library:

¹⁸ The string handling functions of STANDARD are restricted to strings with a maximum length of 255 characters.

Function	Short description	Source code example
<code>brsstrcat()</code>	Appends the contents of the source string (specified as a pointer) to the existing contents of the destination string (also specified as a pointer). ¹⁹ <code>NextAddress</code> is the address of the memory cell that follows the last copied character. ²⁰	<pre>NextAddress := brsstrcat(ADR(DestString), ADR(SourceString));</pre>
<code>brsstrcpy()</code>	Copies the contents of the source string (specified as a pointer) to the destination string (also specified as a pointer). <code>NextAddress</code> is the address that follows the last element described.	<pre>NextAddress := brsstrcpy(ADR(DestString), ADR(SourceString));</pre>
<code>brsstrcmp()</code>	Compares two character strings. If the specified strings are equal, this is displayed with <code>Result=0</code> .	<pre>Result := brsstrcmp(ADR(DestString), ADR(SourceString));</pre>



Both `brsstrcat()` and `brsstrcpy()` do **NOT** check²¹ whether the memory of the source string is sufficient to hold the data!
The user is responsible to ensure sufficient size of the target memory. Failure to do so will result in overwriting memory areas that no longer belong to `DestString` and thus memory access violations.



The corresponding functions for handling WSTRINGS can be found in the `AsBrWStr` library.

Exercise: Working with string functions

Create multiple string variables of different lengths and initial values for the following exercises.

Copying

- 1) Copy the contents of one string to another.
- 2) Try this despite the fact that the destination string might not have sufficient memory. Where or how can it be determined that the string length is beyond the limit?
- 3) Adjust the program so that copying is only performed if the destination string is large enough.
- 4) Try to find a way to copy only as many characters from the source string as actually have room in the destination string.

Concatenating

- 1) Repeatedly append the contents of one string (or a fixed string) to the contents of another string. Observe the repeated appending either in the Watch window or in the Debugger.
- 2) Observe what happens when the destination string is full and consider what risks may result.
- 3) Can you find a possibility in the source code to prevent appending beyond the limits of the destination string?

¹⁹ The end of the existing contents is defined by the first termination character "\0" in the destination string.

²⁰ If the destination string is not large enough, this address is outside the memory area of the destination string!

²¹ Safe functions for string and memory operations are available in ANSI C.

Comparing

- 1) Compare the contents of two strings and find out what the return value of `brsstrcmp()` means.
- 2) Create source code to find out if the extension of a filename (stored in a string) is ".txt". If yes, the filename should be replaced by ".dat".



For strings with a maximum length of 255 characters, there are some useful functions in the STANDARD library. They can be used, for example, to search, replace or position specific characters or words within a string.

Optional exercise: Converting and editing strings

The two variables `DescriptionSTRING` and `DescriptionWSTRING` should generate a message about the current oven temperature `Temperature`:

"The current temperature of the oven is -xx- degrees."

- 1) Add the `AsBrStr` and `AsBrWStr` libraries to the project.
- 2) Assign to both strings the beginning of the message: "The current temperature of the oven is ". Either the assignment operator or a suitable copy function for the respective string type can be used for this in ST.
- 3) Converting to `STRING` or `WSTRING`

Convert the value of the variable "Temperature" into a `STRING` or `WSTRING` and append the information to the existing strings respectively. The conversion functions required for this can be found in the `AslecCon` library. Use the `CONCAT()` function to combine the strings and the `brwscat()` function to combine `Wstrings`.

- 4) Now append the end of the message "degrees" to both strings.
- 5) View the result in the Watch window when changing the value of "Temperature".



Careless or incorrect use of string functions may result in memory access violations when reading or writing beyond the limits of a string.

4.3 Operations on memory areas

In addition to the functions `brsstrcat()`, `brsstrcmp()` and `brsstrcpy()`, which are specifically intended for use with strings, there are other functions that can be used on arbitrary memory areas using the addresses of the memory areas, regardless of the data type.

These functions are also available in the `AsBrStr` library but are independent of the data type of the data to be manipulated.

As with strings, memory areas can be initialized with certain values, data can be copied or moved or the contents of memory areas can be compared.

```
VAR
  DestAddress : UDINT;
  SourceAddress : UDINT;
  NextAddress : UDINT;
  Length : UDINT;
  Value : USINT;
  Result : DINT;
END_VAR
```

The following table provides a short overview of the functions available for these use cases from the `AsBrStr` library:

Function ²²	Short description	Source code example
<code>memset()</code>	Writes <code>NumberBytes</code> times the value of <code>Value</code> in memory beginning with the start address <code>DestAddress</code> . The return value is the address following the last byte written.	<pre>NextAddress := brsmemset(DestAddress, Value, NumberBytes);</pre>
<code>memcpy()</code>	Copies the contents of <code>Length</code> bytes in memory starting from <code>SourceAddress</code> to the memory cells following <code>DestAddress</code> . The return value is the address following the last byte written.	<pre>NextAddress := brsmemcpy(DestAddress, SourceAddress, Length);</pre>
<code>memmove()</code>	Like <code>memcpy()</code> but the memory areas may overlap.	<pre>NextAddress := brsmemmove(DestAddress, SourceAddress, Length);</pre>
<code>memcmp()</code>	Compares the content of <code>Length</code> memory cells that follow the two transferred memory locations <code>DestAddress</code> and <code>SourceAddress</code> byte by byte ²³ .	<pre>Result := brsmemcmp(DestAddress, SourceAddress, Length);</pre>



Care must be taken to use correct addresses during programming when transferring the address parameters to the functions. The functions themselves cannot perform any checks in this regard.

Task: Initializing and transferring memory

In the following two tasks you will initialize memory and implement a FIFO buffer.

Initializing memory

- 1) Initialize the values of all elements with 0 using `brsmemset()` for your existing `PizzaList`.
- 2) Can you find a way to initialize the elements of the `PizzaList` not with 0, but with defined values of a "Reset-Pizza"? What problems occur here when using `brsmemset()`?

Transferring to memory: FIFO buffer

The value change of the `State` variable of a state machine should be recorded in an array with 20 elements. Each time the value is changed, all elements in the array are moved back one position and the new value is saved on index 0.

- 1) Create the state variable `State` of type `StateEnum` (INIT, IDLE, STOPPED, RUN).
- 2) Create the array in which the value of the state variable can be recorded.
- 3) Program the FIFO function as described above. If the array is full, the oldest value in the buffer is discarded.

For operations on memory areas, regardless of whether this memory area is realized by a large static array or by dynamic memory (see 5.1 "Dynamic memory" on page 48), care must be taken during programming to ensure correct management of the used and available memory via the pointers used. Otherwise, unwanted overwriting of needed data or even memory access violations will occur.

²² All functions have the prefix `brs`, i.e. `brsmemset()`, etc.

²³ This can lead to cycle time violations if the memory area to be compared is too large.

Compared to using dynamic memory, using static arrays has the advantage that there is no need to worry about correctly reserving and freeing up the memory block. Dynamic memory management is possible. In order to prevent often difficult and lengthy troubleshooting, the use of static variables is preferable.

4.4 Error handling with IEC Check

When using arrays and strings, programming errors can have serious consequences if the memory access violations mentioned above occur.

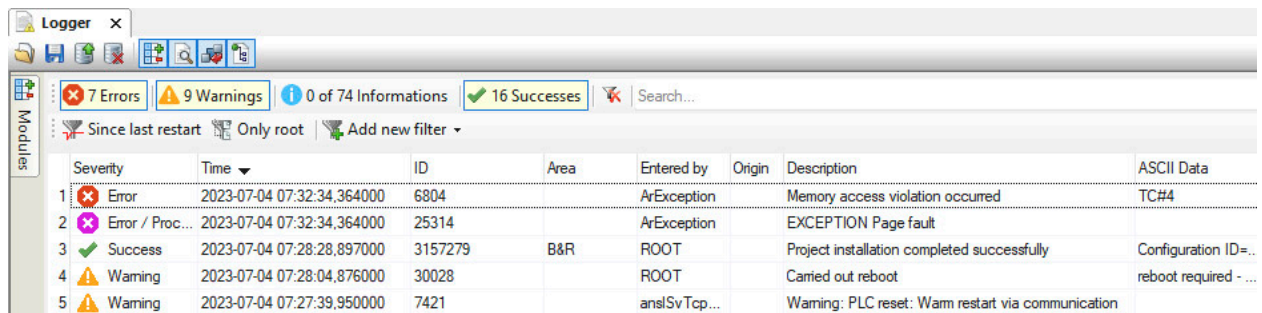
4.4.1 Causes of errors

EXCEPTION page fault

If memory access violations occur due to impermissible indices or the use of strings that are too long and write beyond the declared string limits, this may cause the system to restart in SERVICE mode.

The use of dynamic variables that have not yet been initialized is also an error that places the system in SERVICE mode.

The system crash and restart is in all cases caused by a page fault, which is entered in the Logger.



Severity	Time	ID	Area	Entered by	Origin	Description	ASCII Data
1 Error	2023-07-04 07:32:34.364000	6804		ArException		Memory access violation occurred	TC#4
2 Error / Proc...	2023-07-04 07:32:34.364000	25314		ArException		EXCEPTION Page fault	
3 Success	2023-07-04 07:28:28.897000	3157279	B&R	ROOT		Project installation completed successfully	Configuration ID=...
4 Warning	2023-07-04 07:28:04.876000	30028		ROOT		Carried out reboot	reboot required - ...
5 Warning	2023-07-04 07:27:39.950000	7421		ansISvTcp...		Warning: PLC reset: Warm restart via communication	

Figure 29: Logger entry after a page fault

The problem is that an EXCEPTION page fault can have many causes that are not listed with separate error numbers. Better ways to localize the causes need to be explored to find the bugs in the code. A tool for this is "Backtrace" from the Logger.

Undetected memory access violations

An even bigger problem than the EXCEPTION page fault described above can be undetected memory access violations. For example, if writing beyond the valid limits of an array or string does not result in a page fault, but "only" overwrites the value of an "adjacent" variable in memory.



Shown here is an example of a simple state machine whose state variable has been declared with an enumerated data type.

Declarations

```
TYPE
  StatesEnum:
    (
      INIT,
      WAIT,
      MOVE_UP,
      MOVE_DOWN,
      ERROR
    );
END_TYPE

VAR
  ArrayTricky : ARRAY[0..5] OF DINT;
  State : StatesEnum := INIT;
  index : DINT := 0;
  Trigger : DINT := 0;
  Info : STRING[40];
END_VAR
```

The enumerated type StatesEnum has 5 elements whose corresponding values are 0-4. The element Error thus represents the value 4.

The last line in the program code is the statement that causes an impermissible write access by setting an index that is outside the limit.

Alternatively, the last line of the program code could read:

```
ArrayTricky[index] := ERROR;
```

Program code

```
CASE State OF
  INIT:
    Info := 'Init';
    IF Trigger = 1 THEN
      State := WAIT;
    END_IF
  WAIT:
    Info := 'Wait';
    IF Trigger = 2 THEN
      State := MOVE_UP;
    END_IF
  MOVE_UP:
    Info := 'MoveUp';
    IF Trigger = 3 THEN
      State := MOVE_DOWN;
    END_IF
  MOVE_DOWN:
    Info := 'MoveDown';
    IF Trigger = 4 THEN
      State := WAIT;
    END_IF
  ERROR:
    Info := 'Error';
    IF Trigger = 0 THEN
      State := INIT;
    END_IF
END_CASE

ArrayTricky[index] := 4;
```

For the given declaration sequence, the variables are stored in memory as shown in the figures below. For clarity, four memory cells are always grouped together in a row in the memory display. A DINT uses 32 bits and thus 4 memory cells.

Permitted access (index=2)

address	memory cell	name of variable
0x0802 BB28	\0	
	\0	
	t	
	i	
	a	
0x0802 BB14	W	Info
0x0802 BB10	1	Trigger
0x0802 BB0C	2	index
0x0802 BB08	WAIT	State
	0	ArrayTricky[5]
0x0802 BB00	0	ArrayTricky[4]
	0	ArrayTricky[3]
0x0802 BAF8	4	ArrayTricky[2]
	0	ArrayTricky[1]
0x0802 BAF0	4	ArrayTricky[0]

ArrayTricky[2] := 4;

Access outside array limits (index=6)

address	memory cell	name of variable
0x0802 BB28	\0	
	r	
	o	
	r	
	r	
0x0802 BB14	E	Info
0x0802 BB10	1	Trigger
0x0802 BB0C	6	index
0x0802 BB08	ERROR	State
	0	ArrayTricky[5]
0x0802 BB00	0	ArrayTricky[4]
	0	ArrayTricky[3]
0x0802 BAF8	4	ArrayTricky[2]
	0	ArrayTricky[1]
0x0802 BAF0	4	ArrayTricky[0]

ArrayTricky[6] := 4;

As can be seen, a write access to `ArrayTricky[6]` beyond the array limits causes the `State` variable to be overwritten. As a result, the manufacturing system, whose state machine is affected here, goes into an error state even though there is no technical cause for it.

A programming error is the only cause. Finding the error could be difficult, however, because the invalid access does not show up in the Logger either.

The behavior shown in the figures can also be observed in the Watch window of the program. Permitted memory access again on the left, access outside array limits on the right.

Name	Type	Value
State	StatesEnum	WAIT
Trigger	DINT	1
Info	STRING[40]	'Wait'
ArrayTricky	DINT[0..5]	
ArrayTricky[0]	DINT	4
ArrayTricky[1]	DINT	0
ArrayTricky[2]	DINT	4
ArrayTricky[3]	DINT	0
ArrayTricky[4]	DINT	0
ArrayTricky[5]	DINT	0
index	DINT	2

Figure 30: Values of the variables in the Watch window when index=2

Name	Type	Value
State	StatesEnum	ERROR
Trigger	DINT	1
Info	STRING[40]	'Error'
ArrayTricky	DINT[0..5]	
ArrayTricky[0]	DINT	4
ArrayTricky[1]	DINT	0
ArrayTricky[2]	DINT	4
ArrayTricky[3]	DINT	0
ArrayTricky[4]	DINT	0
ArrayTricky[5]	DINT	0
index	DINT	6

Figure 31: Values of the variables in the Watch window when index=6

Other "silent" errors

Also, assigning values to variables that are outside the defined range of the enumerated data type or subrange is not automatically detected and prevented by the system. Again, however, an invalid value assignment can cause the application to behave incorrectly.

4.4.2 IEC Check library

The functions of the IEC Check library provide the possibility to localize the mentioned memory access violations at runtime. EXCEPTION page faults, as well as limit or value range overshoots and undershoots of data types with sub-ranges can be detected.

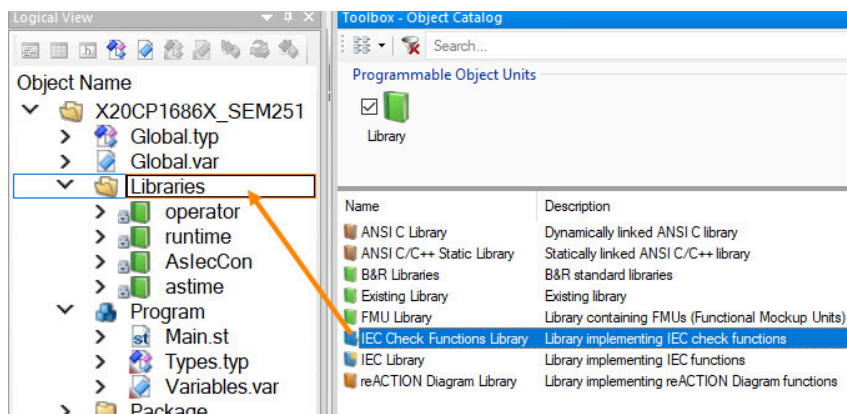


Figure 32: Adding the IEC Check library

The library is inserted into the Logical View of the project via the Toolbox and already features implemented functions. The names and parameters of the functions cannot be changed, the implementation of each contained function can or should be adapted to the respective needs.



IEC Check should only be used for troubleshooting in exceptional cases and only on test machines or monitored machines. Some errors cannot be found in a simulation because they do not occur there.

Due to the operator overload that is executed in the background, a significant amount of system resources is required!



The use of the library in the project can be disabled again at any time with the compiler option `-D _IGNORE_CHECKLIB`.

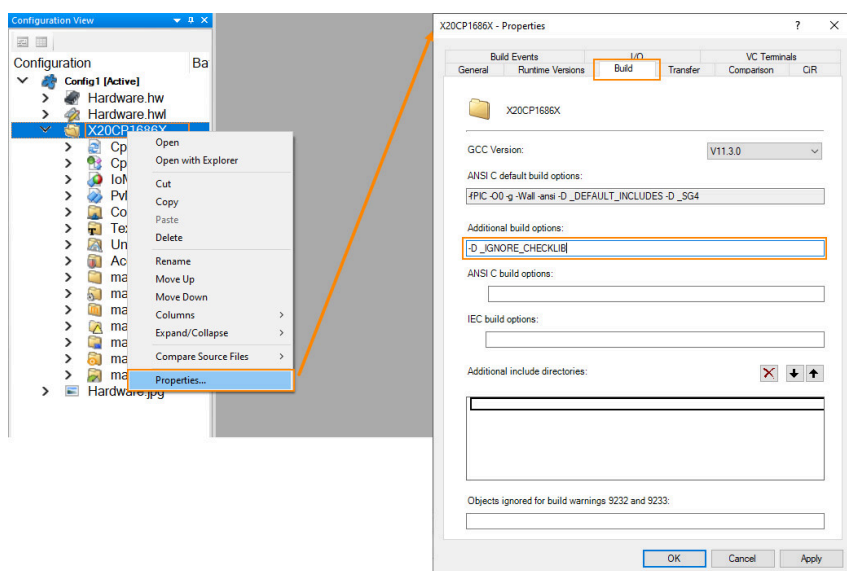


Figure 33: Setting compiler options in the project

Right-click on the CPU folder in the Configuration View to open the properties window.

You can enter additional compiler options in the "Compile" tab.

Function	Description	What is intercepted?
CheckBounds()	Checks for limit overshoots in the user project.	Invalid indices for arrays are reported, string limit overshoots are NOT intercepted!
CheckRange()	Checks for limit overshoots during write access to enum variables.	Invalid values for enum variables are reported ²⁴ .

Table 12: Overview of IEC Check library functions

²⁴ Even without IEC Check, an invalid value assignment to an enum variable can and should be intercepted by appropriate error handling in a DEFAULT case.

Function	Description	What is intercepted?
CheckSubrange()	Checks for an invalid value assignment to a subrange variable.	Invalid values for variables with a subrange ; value is not written and is set to the lower or upper limit of the subrange.
CheckDiv()	Checks for invalid divisions.	Division by 0
CheckReadAccess()	Checks the addresses during read access with dynamic memory access.	This function takes effect if a reference does not yet have an address assignment. Invalid read access with references that are initialized is not intercepted.
CheckWriteAccess()	Checks the addresses during write access with dynamic memory access.	Write access to references that have not yet been initialized is prevented. Invalid write access to references that are initialized is not intercepted.

Table 12: Overview of IEC Check library functions

 [Programming \ Libraries \ IEC Check library](#)

[Programming \ Libraries \ Configuration, system information, runtime control \ SYS_Lib \ Functions and function blocks \ Error handling](#)

By adding the IEC Check library - without any further configuration of the functions defined there - the error generated for the invalid write attempt of

```
ArrayTricky[6] := 4;
```

is entered in the logbook.

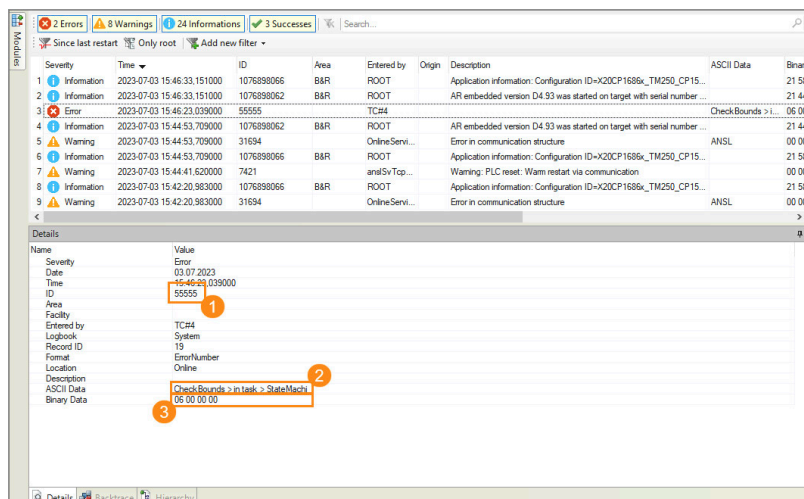


Figure 34: Logger entry when using the IEC Check library

Due to the default implementation of the function `CheckBounds()`, the controller is simultaneously set to SERVICE mode.

- 1) The default error number of all functions from IEC Check is 55555. This can (and should) be adjusted when implementing the function.
- 2) The ASCII data indicates the task where the error can be found and IEC Check function that has been triggered.
- 3) The binary data indicates which value caused the limit overshoot. 0x06 corresponds exactly to index 6, which generates the error.

`CheckBounds()` puts the system into SERVICE mode due to the call of the function `ERRxfatal()`. By adapting the implementation and using `ERRxwarning()` instead of `ERRxfatal()`, you can ensure that only a warning is entered in the Logger instead of a system restart.

Error localization with Backtrace in the Logger

By adapting the implementation of `MakeEntry()` accordingly, precise error localization in the code can be achieved. `MakeEntry()` is the function that generates a corresponding Logger entry for all IEC Check functions. In `MakeEntry()`, calling the `ERRxfatal()` function is replaced by:

```
ERRxwarning(number,index,ADR(out_text));
brsmemset(0,0,1);
```

With the parameters for `brsmemset()` set in this way, a page fault is now deliberately caused when `MakeEntry()` is called.

Severity	Time	ID	Entered by	Description
1 Error	2024-02-07 13:5...	6803	ArException	Stack underflow in task
2 Error / Processor Exception	2024-02-07 13:5...	25314	ArException	EXCEPTION Page fault
3 Warning	2024-02-07 13:5...	55555	TC#4	CheckBounds > in tas
4 Warning	2024-02-07 13:5...	7421	ansisV/cpre...	Warning: PLC reset: VV...
5 Success	2024-02-07 13:5...	3157279	ROOT	Project installation com... Configuration ID="BRt

Backtrace

Name

Backtrace Data

- Name: "brsftoa" / Address: 0x041283d0 / Call Address: 0x041285b7 / Param Count: 4 / Parameter: 0x00000000, 0x00000000, 0x00000000, 0x00000000
- EXCEPTION POSITION: Module: "AsBrStr" / Offset: 0x000004eb
- CALLSTACK POSITION: Module: "AsBrStr" / Offset: 0x000001e7
- Name: "brsmemset" / Address: 0x04128590 / Call Address: 0x0413230c / Param Count: 4 / Parameter: 0x00000000, 0x00000000, 0x00000000, 0x00000000
- FUNCTION START POSITION: Module: "AsBrStr" / Offset: 0x000001c0
- CALLSTACK POSITION: Module: "lecCheck" / Offset: 0x000008cc
- Name: "CheckBounds" / Address: 0x0413230c / Call Address: 0x04131ebb / Param Count: 4 / Parameter: 0x0000d903, 0x00000000, 0x00000000, 0x00000000
- FUNCTION START POSITION: Module: "lecCheck" / Offset: 0x000008cc
- CALLSTACK POSITION: Module: "lecCheck" / Offset: 0x0000047b
- Name: "MakeEntry" / Address: 0x04131ebb / Call Address: 0x0434bb3b / Param Count: 4 / Parameter: 0x00000006, 0x00000000, 0x00000000, 0x00000000
- FUNCTION START POSITION: Module: "lecCheck" / Offset: 0x0000047b
- CALLSTACK POSITION: Module: "StateMachi" / Offset: 0x00000033
- Name: "BUR_ENTRY_CYCLIC_FUNC" / Address: 0x0434bb3b / Call Address: 0x00573912 / Param Count: 4 / Parameter: 0x00573912, 0x00573912, 0x00573912, 0x00573912
- FUNCTION START POSITION: Module: "StateMachi" / Offset: 0x00000033
- Address: 0x00573912 / Call Address: 0x00573624 / Param Count: 4 / Parameter: 0x0434b300, 0x00000000, 0x069eff20, 0x004f

- 1) Warning that is entered in the Logbook by `ERRxwarning()`.
- 2) Logger entry via `brsmemset(0,0,1)`.
- 3) View of the Backtrace data for the selected EXCEPTION page fault.
- 4) Trace back to the source code line that is causing the error by double-clicking on the entry.



Error numbers that can be assigned by users range from 0xC350 to 0xEA5F (50 000 – 59 999). All other numbers are already assigned or reserved for future functions.

Using the return values of the IEC Check functions

With the functions `CheckBounds()`, `CheckRange()`, `CheckSubrange()` and `CheckDiv()`, the return value of the functions is used in the program at the location where it is checked.

In the case of `CheckBounds()`, there is no access outside the valid array limits and the valid lower (first) or upper (last) array element is accessed.

If the index used is within the valid limits, that value is, of course, used (ELSE case).

With problematic access:

```
ArrayTricky[6] := 4;
```

```
FUNCTION CheckBounds
  IF index < lower THEN
    CheckBounds := lower;
    MakeEntry(55555, index, 'CheckBounds');
  ELSIF index > upper THEN
    CheckBounds := upper;
    MakeEntry(55555, index, 'CheckBounds');
  ELSE
    CheckBounds := index;
  END_IF
END_FUNCTION
```

The invalid index 6 is now set by `CheckBounds()` to the last valid value for the array – i.e. 5 – and overwriting the variable `State` from the example is thus prevented.

Assigning the function's return value to the checked variables in the source code is independent of the Logger entry created by `MakeEntry()`.

The functions `CheckRange()` and `CheckSubrange()` assign the upper or lower valid value to the checked variable. By default, `CheckDiv()` executes a division by 1 instead of division by 0.

Exercise: Using IEC Check in the project

1) Accessing outside the array limits

Access an element outside the valid array limits on an array used in the program (e.g. `PizzaList`). Observe the behavior in the Watch window and in the Logger.

2) Adding the IEC Check library

Now add the IEC Check library in the Logical View under Libraries. Transfer your project again after a rebuild (using "Rebuild all") and execute unauthorized access outside the array limits again.

Analyze the message appearing in the Logger and the system behavior.

Optional exercise: Individually customizing IEC Check functions

Change the standard implementation of the function `MakeEntry()` from the IEC Check library so a page fault is deliberately caused when `MakeEntry()` is called.

In the event of access outside the valid array limits, analyze the Backtrace data for the Logger entry and try to locate the exact cause in the source code.

4.4.3 Limitations of IEC Check

Using the functions from the IEC Check library can, in many cases, give an indication of where the cause of an error might be.

However, not all errors can be found or intercepted even with IEC Check. The following list provides a few examples where even IEC Check cannot detect or localize the error (cause) more precisely, so special care must be taken when programming.

- Writing beyond string limits cannot be detected.
- Invalid access with references that are initialized cannot be intercepted. `CheckWriteAccess()` and `CheckReadAccess()` only check for invalid, i.e. uninitialized, references.
- Overwriting values of local variables with pointers from other tasks remains undetected.
- Detecting a division by zero with `CheckDiv()` may fail when using non-integer data types, since floating-point numbers can only ever be approximated.
- Adjustments to the standard implementation of the IEC Check functions are necessary for exact error localization.



The system load increases significantly when using IEC-Check because the functions of the library are executed at each relevant execution of an instruction of the program.

Every time an array element is accessed (read or write), for example, the function `CheckBounds()` is always called and executed first.

IEC Check should therefore only be used for debugging and not during productive operation of a system.

5 Advanced memory management

5.1 Dynamic memory

It is possible not to allocate memory in static variables for the entire runtime, but to request it dynamically at runtime and to free it up again.

Dynamic memory can be used for reading files as well as for Ethernet communication, if it is only known at runtime how much memory is actually required.



Ready-made B&R libraries are available for almost all applications, so in most cases it is no longer necessary to manually reserve dynamic memory.

Using the user-specific B&R libraries is recommended, since many critical errors can occur when reserving memory manually, and these errors are often very difficult to find.



Once memory is reserved by a function, the programmer is responsible for freeing it up again using the corresponding function!

Failure to do so results in memory leaks, which can eventually consume the available memory.

5.1.1 Reserving memory blocks

Sometimes it may be necessary to reserve a memory area in DRAM at runtime. The serial interface of a controller operates with dynamic memory, for example, which it requires at runtime to maintain error-free communication.

Dynamically reserved memory is available to the user in the program. It can always be accessed via the start address of the reserved memory. The reserved memory area is no longer available to the system until it is freed up again by the user.

The memory being reserved must comprise consecutive available memory locations on the system in order for it to be used by the application program.



The contents of dynamically requested memory must be initialized by the user via a reference of the appropriate data type. Since the contents of memory reserved are in DRAM, it will be lost each time the controller is restarted.

Reserving and freeing up dynamic memory is possible via functions and function blocks from the SYS_Lib and AsMem libraries. These functions should only be called in the program's init and exit subroutines.

5.1.2 Managing memory with AsMem

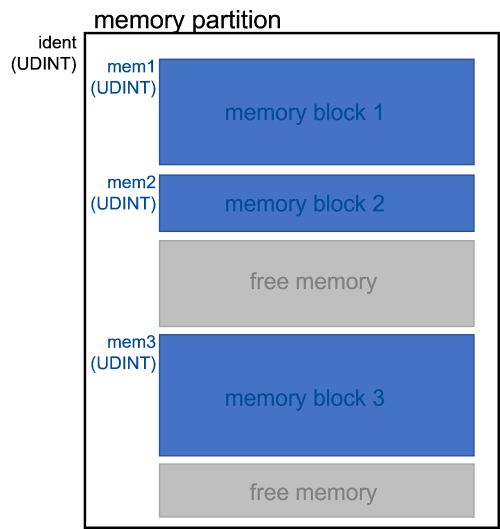
The AsMem library can be used in the INIT subroutine of a program to reserve a large partition of memory on the system. Function blocks in the cyclic program section can be used to divert memory blocks from this memory partition and then free it up again. The term "partition" does not refer to a partition in the sense of "drive" (C: or D:) in the flash memory but to a contiguous (large) memory block in DRAM.

The AsMem library contains function blocks that can be used to dynamically manage even large amounts of memory from DRAM at runtime. These are used, for example, when loading files from a USB device so that they can be evaluated by an application.

Function block	Description
AsMemPartCreate	Creating a memory partition; must be done in INIT
AsMemPartDestroy	Freeing up a memory partition; must be done in EXIT
AsMemPartAlloc	Reserving a memory block within the partition
AsMemPartAllocClear	Reserving a memory block within the partition; contents of the block are initialized with 0
AsMemPartRealloc	Increasing or decreasing the size of the existing memory block within the partition
AsMemPartFree	Freeing up the memory block
AsMemPartInfo	Provides information about available memory in an allocated partition, including information about the largest remaining block of free memory in the partition.

Table 13: Function blocks from AsMem

Using the function blocks



A partition with the desired size must first be allocated with AsMemPartCreate(). This is only permitted in the initialization subroutine and not in the cyclic section of the program.

In the cyclic section of the program, the functions AsMemPartAlloc(), AsMemPartAllocClear(), AsMemPartRealloc() and AsMemPartFree() can then be used to request, resize and free up one or more memory blocks from this partition.

A partition is identified by the "ident" value that is returned when the partition is created. This is a unique identification number of the partition that is assigned by the system.

A reserved memory block is accessed via its start address. The start addresses are returned by the allocation blocks AsMemPartAlloc(), AsMemPartAllocClear() and AsMemPartRealloc().

Figure 35: Managing memory with AsMem

For each reserved memory block there is a memory overhead of 8 bytes to manage the block. The minimum size of a memory block is 16 bytes; if less is requested, 16 bytes + 8 bytes of memory are still reserved within the partition.

Reading and writing data from or to reserved memory blocks is done using the already known functions from the AsBrStr library (e.g. brsmemcpy()).

For using the function blocks from the AsMem library, the executable sample project LibAsMem1_ST is available, which can be imported into Automation Studio to show how memory areas in DRAM are dynamically managed. The memory allocated with AsMem is located in DRAM, but encapsulated within the partition. This prevents a memory leak from occurring, which would use up all DRAM memory. In this way, only the memory within the partition can be used up.

[Programming \ Libraries \ Configuration, system information, runtime control \ AsMem](#)

[Programming \ Examples \ Examples - Libraries \ Configuration, system information and runtime control \ Managing memory areas](#)



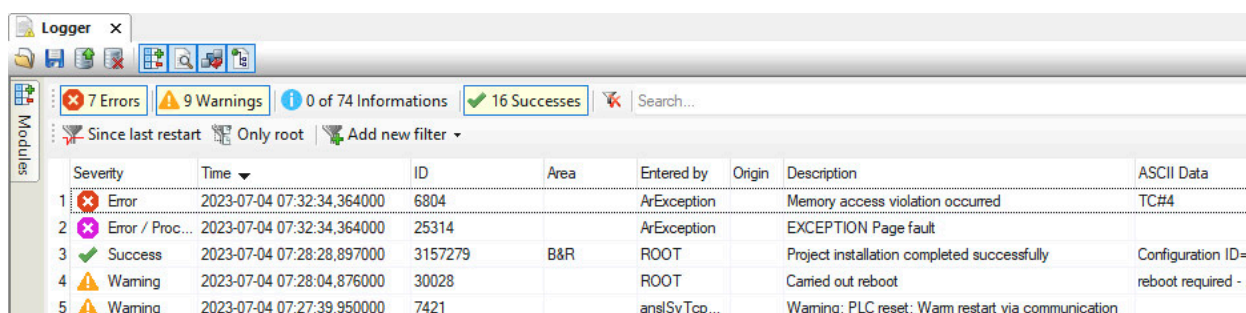
Dynamically reserved memory must also be **freed up again** by the user!

5.1.3 Risks and error sources

If dynamic memory is used carelessly, there are several errors that can occur. These errors often cause problems with Ethernet communication or HMI application. This can also result in a system crash followed by SERVICE mode, however.

Memory leaks

The most common cause is forgetting to free up reserved memory. This can lead to the aforementioned memory leaks. This causes the memory to "overflow" and the system can only restart with the resulting ArException in SERVICE mode. The corresponding message in the Logger looks like this:



The screenshot shows the 'Logger' window with a summary bar indicating 7 Errors, 9 Warnings, 0 of 74 Informations, and 16 Successes. The main table lists the following entries:

Severity	Time	ID	Area	Entered by	Origin	Description	ASCII Data
Error	2023-07-04 07:32:34.364000	6804		ArException		Memory access violation occurred	TC#4
Error / Proc...	2023-07-04 07:32:34.364000	25314		ArException		EXCEPTION Page fault	
Success	2023-07-04 07:28:28.897000	3157279	B&R	ROOT		Project installation completed successfully	Configuration ID=...
Warning	2023-07-04 07:28:04.876000	30028		ROOT		Carried out reboot	reboot required - ...
Warning	2023-07-04 07:27:39.950000	7421		anslSvTcp...		Warning: PLC reset: Warm restart via communication	

Figure 36: Logger entry after a memory leak

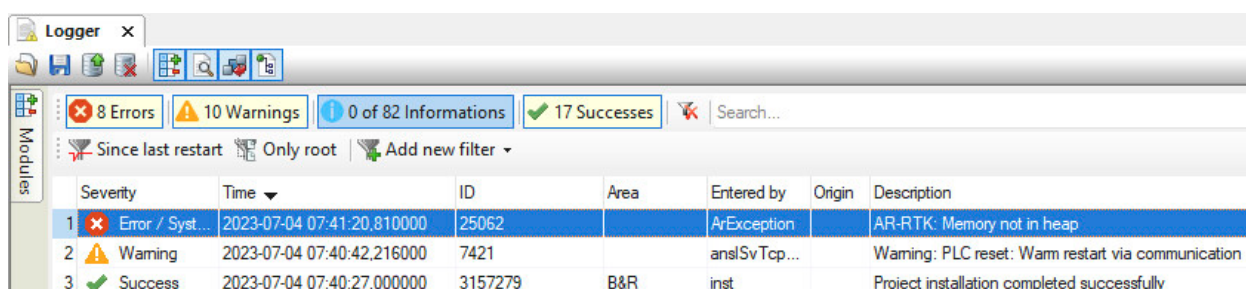
Invalid memory access

The cause of the error here is not incorrectly reserving or failing to release the dynamic memory, but the fact that, when accessing the reserved memory with pointers or references, addresses are used that refer to a memory area that is used by Automation Runtime, for example.

These invalid accesses cannot be intercepted or localized more precisely even with IEC-Check, since they are not static arrays and the pointers or references are already initialized when they are used.

Freeing up already released memory

Trying to free up memory that is no longer reserved will also cause an ArException and put the system into SERVICE mode.



The screenshot shows the 'Logger' window with a summary bar indicating 8 Errors, 10 Warnings, 0 of 82 Informations, and 17 Successes. The main table lists the following entries:

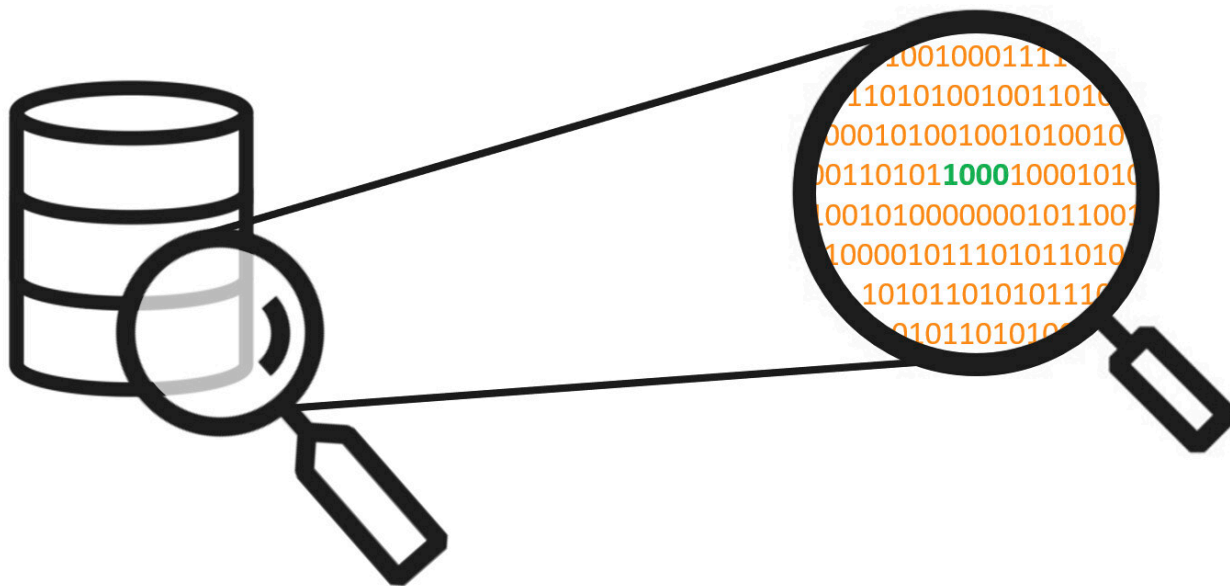
Severity	Time	ID	Area	Entered by	Origin	Description
Error / Syst...	2023-07-04 07:41:20.810000	25062		ArException		AR-RTK: Memory not in heap
Warning	2023-07-04 07:40:42.216000	7421		anslSvTcp...		Warning: PLC reset: Warm restart via communication
Success	2023-07-04 07:40:27.000000	3157279	B&R	inst		Project installation completed successfully

Figure 37: Logger message when trying to free up memory that is no longer reserved

To prevent this scenario, pointers that are released or no longer used should always be set to zero. This makes it possible to avoid these "dangling pointers". Every time a pointer is accessed, it must, of course, be secured with a "<0" check.

6 Summary

Anyone who is responsible for designing a control application is constantly confronted with data. Data is managed by variables and constants, regardless of the programming language. A variety of IEC base data types, arrays and structures are available for this purpose. Automation Studio features declaration editor for easily initializing constants, variables and arrays. Function blocks and functions are provided for managing memory blocks in the application. The basic element of data processing is the storage format of the data on the respective platform. Functions are available for determining and using memory addresses and memory sizes in the program.



Locating errors and eliminating their cause is an important part of developing and maintaining applications.



B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

